HLT Tracking
Specificities and Phase-2 changes

TRACKING PHYSICS OBJECT GROUP

Pietro Grutta[1], Jan Schulz[2]

[1]University of Padua & INFN, Italy
[2]RWTH Aachen University, Germany

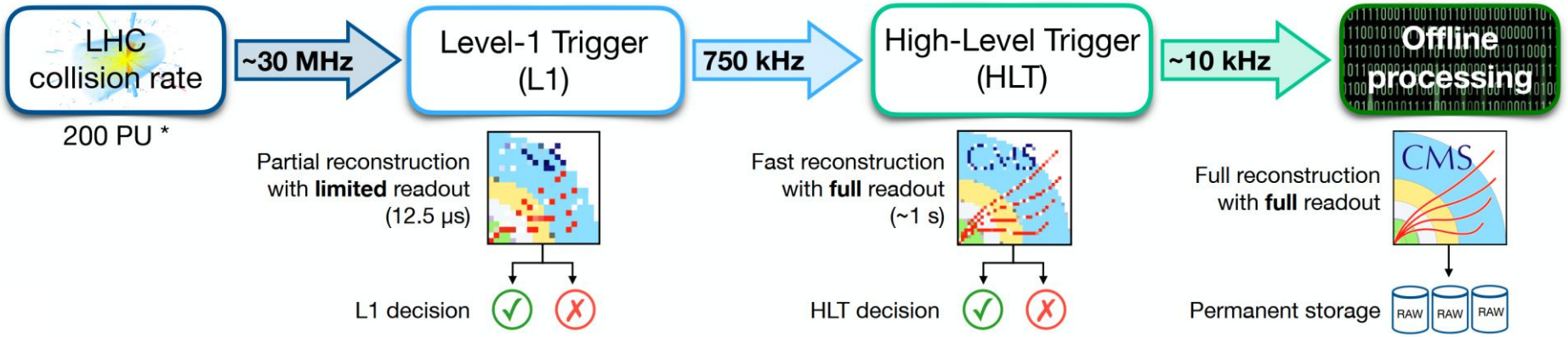
on behalf of the Tracking POG

email: cms-trigger-trk-conveners@cern.ch

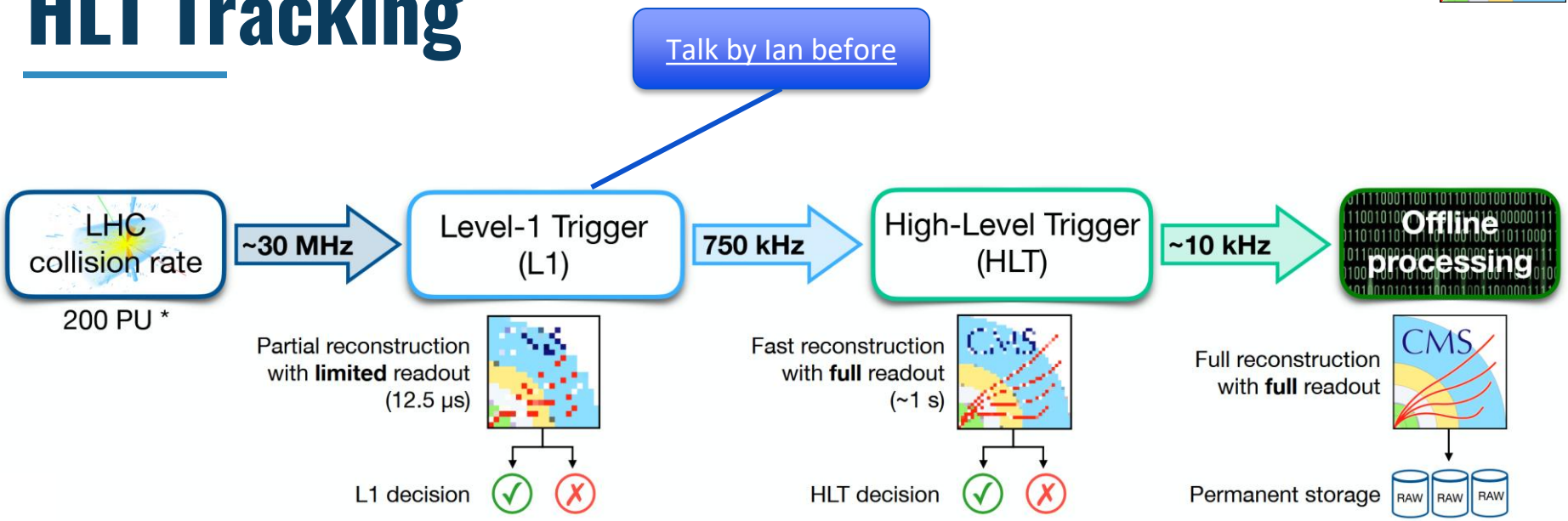
Tracker DPG Workshop: Indico

ULB/VUB Brussels, 16th October 2026

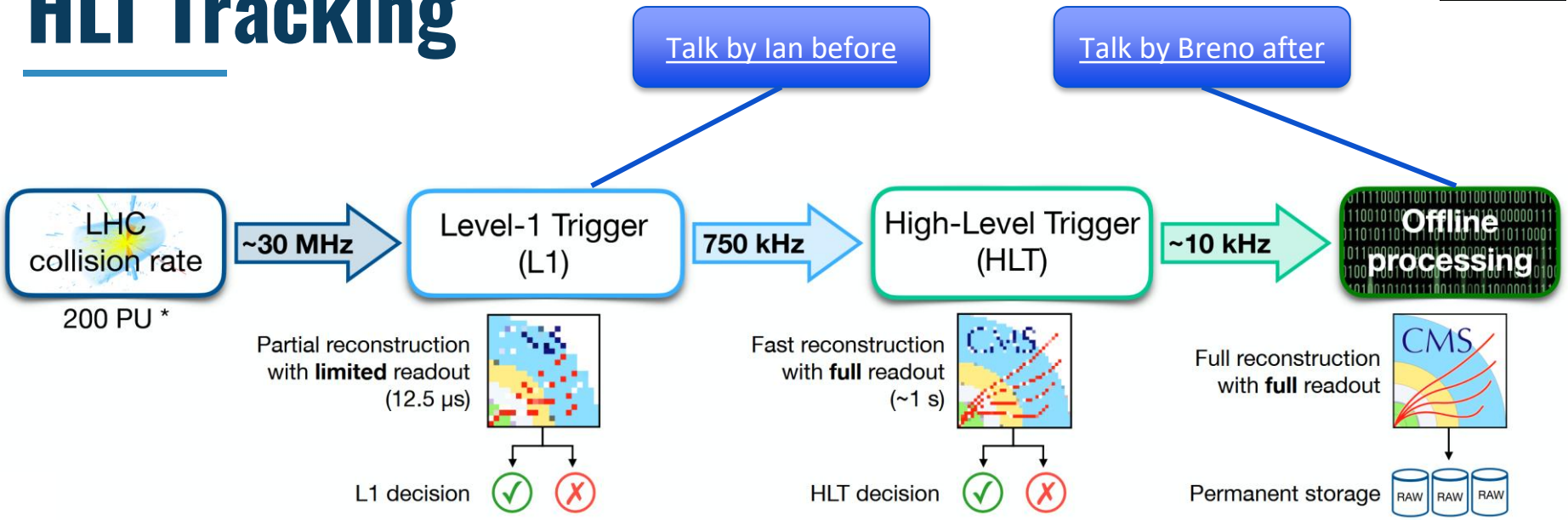
HLT Tracking



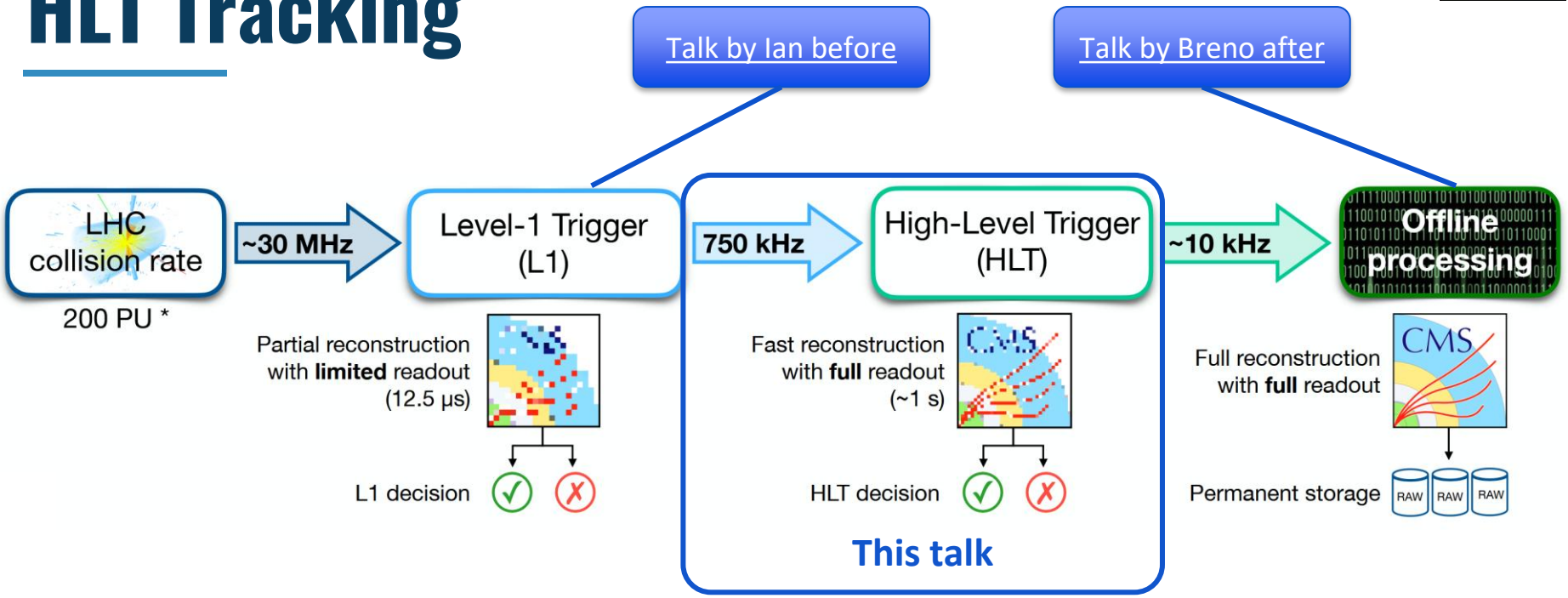
HLT Tracking



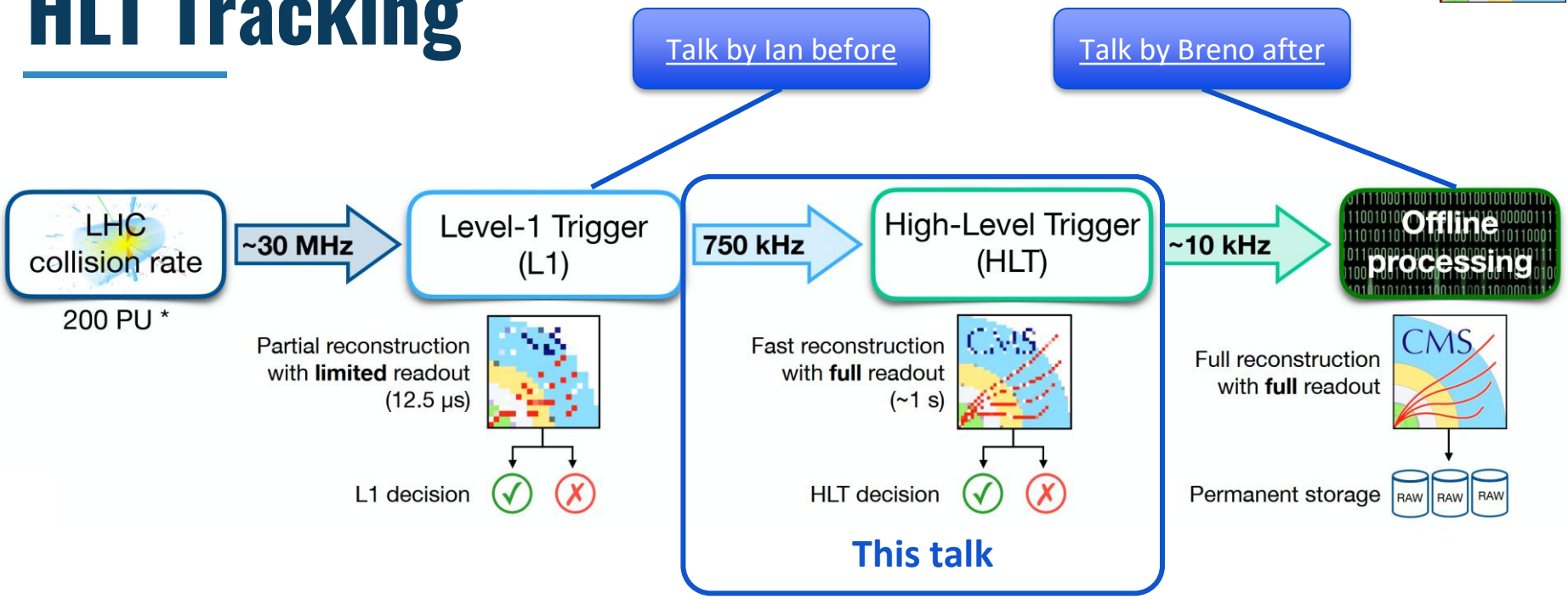
HLT Tracking



HLT Tracking



HLT Tracking



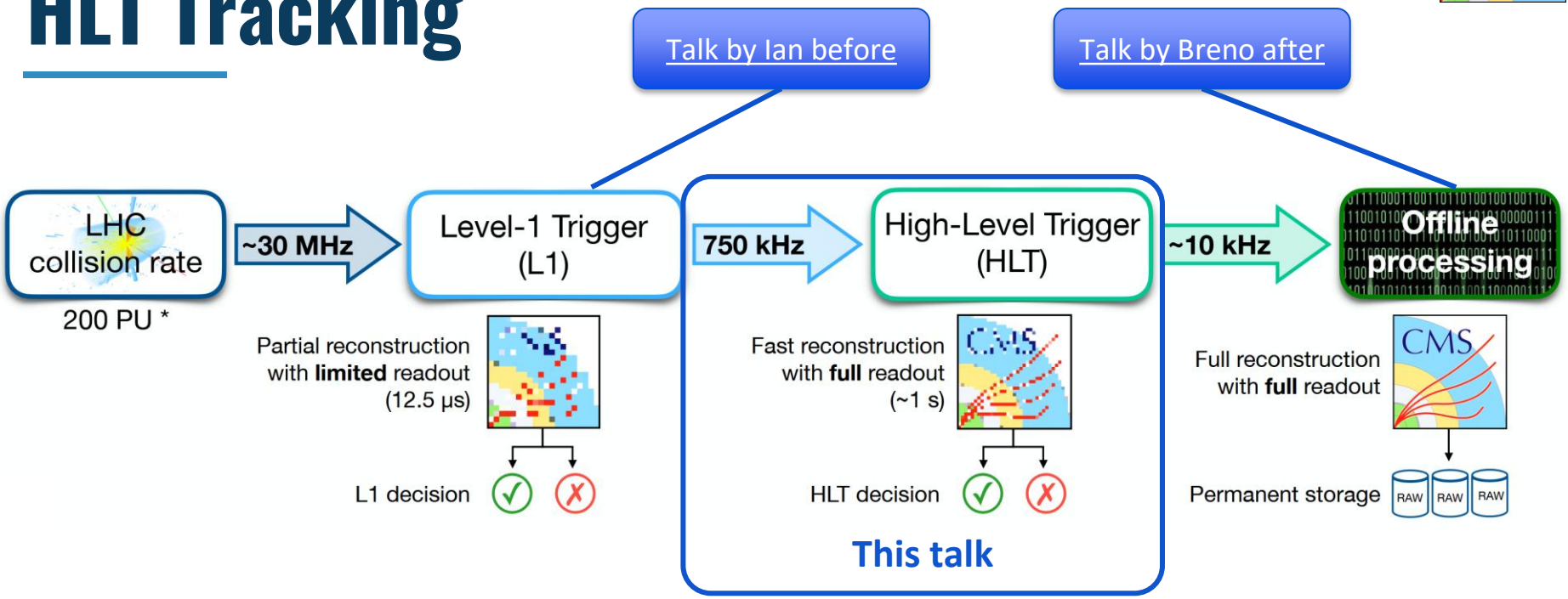
Compared to L1:

- Full Tracker readout available (Inner Tracker!)
- More flexibility

Compared to offline:

- Use same algorithms
- Tight computing/ timing budget

HLT Tracking



HLT selection needs good tracking info as well, whether it's in the form of quick "pixel" tracking or full HLT tracking chain

What we are after:

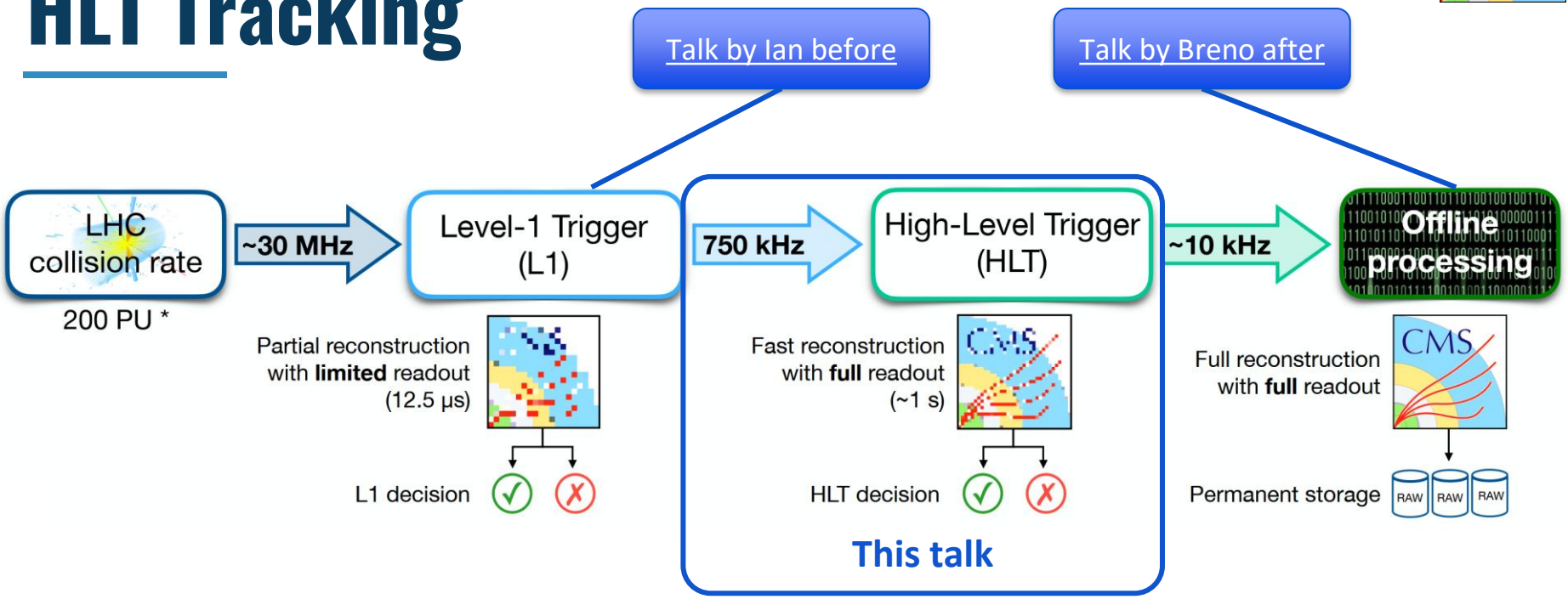
Compared to L1:

- Full Tracker readout available (Inner Tracker!)
- More flexibility

Compared to offline:

- Use same algorithms
- Tight computing/ timing budget

HLT Tracking



HLT selection needs good tracking info as well, whether it's in the form of quick "pixel" tracking or full HLT tracking chain

What we are after:

Physics performance
→ retain or even improve tracking output for optimal event selection + scouting

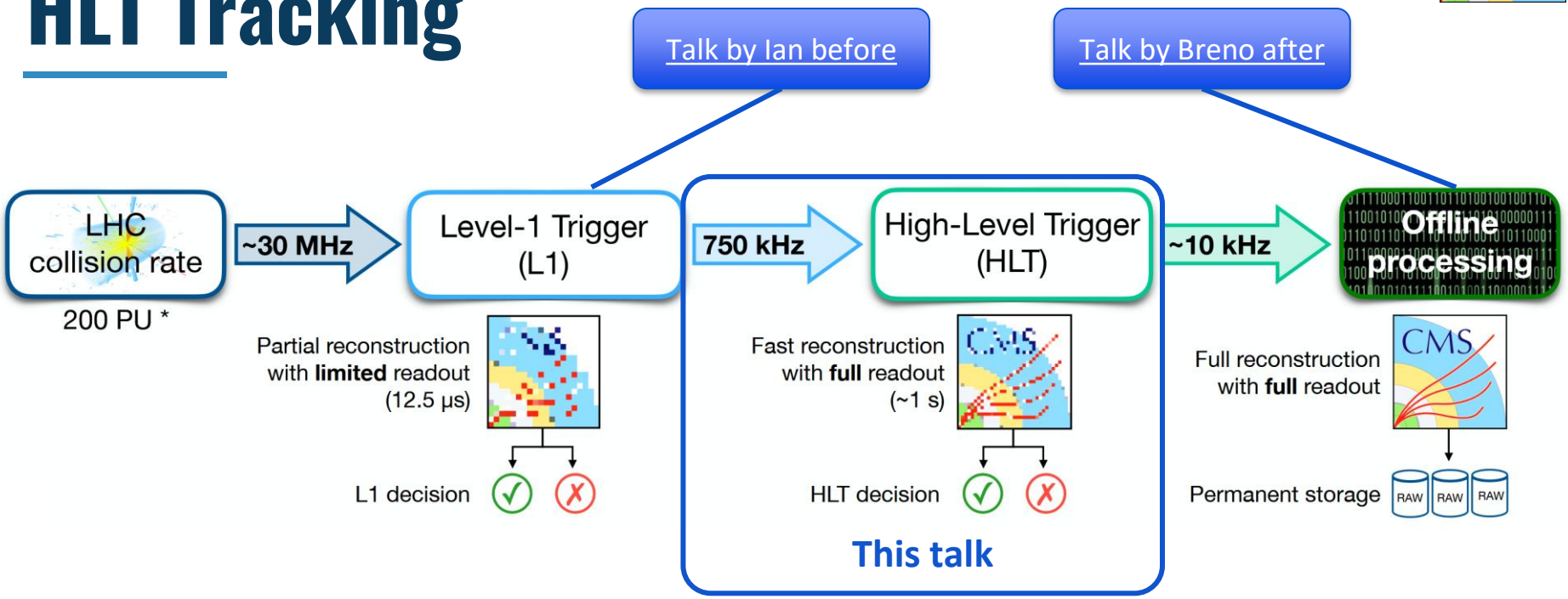
Compared to L1:

- Full Tracker readout available (Inner Tracker!)
- More flexibility

Compared to offline:

- Use same algorithms
- Tight computing/ timing budget

HLT Tracking



HLT selection needs good tracking info as well, whether it's in the form of quick "pixel" tracking or full HLT tracking chain

What we are after:

Physics performance
→ retain or even improve tracking output for optimal event selection + scouting

Computational performance
→ reduce timing for being able to run (more)

Compared to L1:

- Full Tracker readout available (Inner Tracker!)
- More flexibility

Compared to offline:

- Use same algorithms
- Tight computing/ timing budget

Tracing the Past

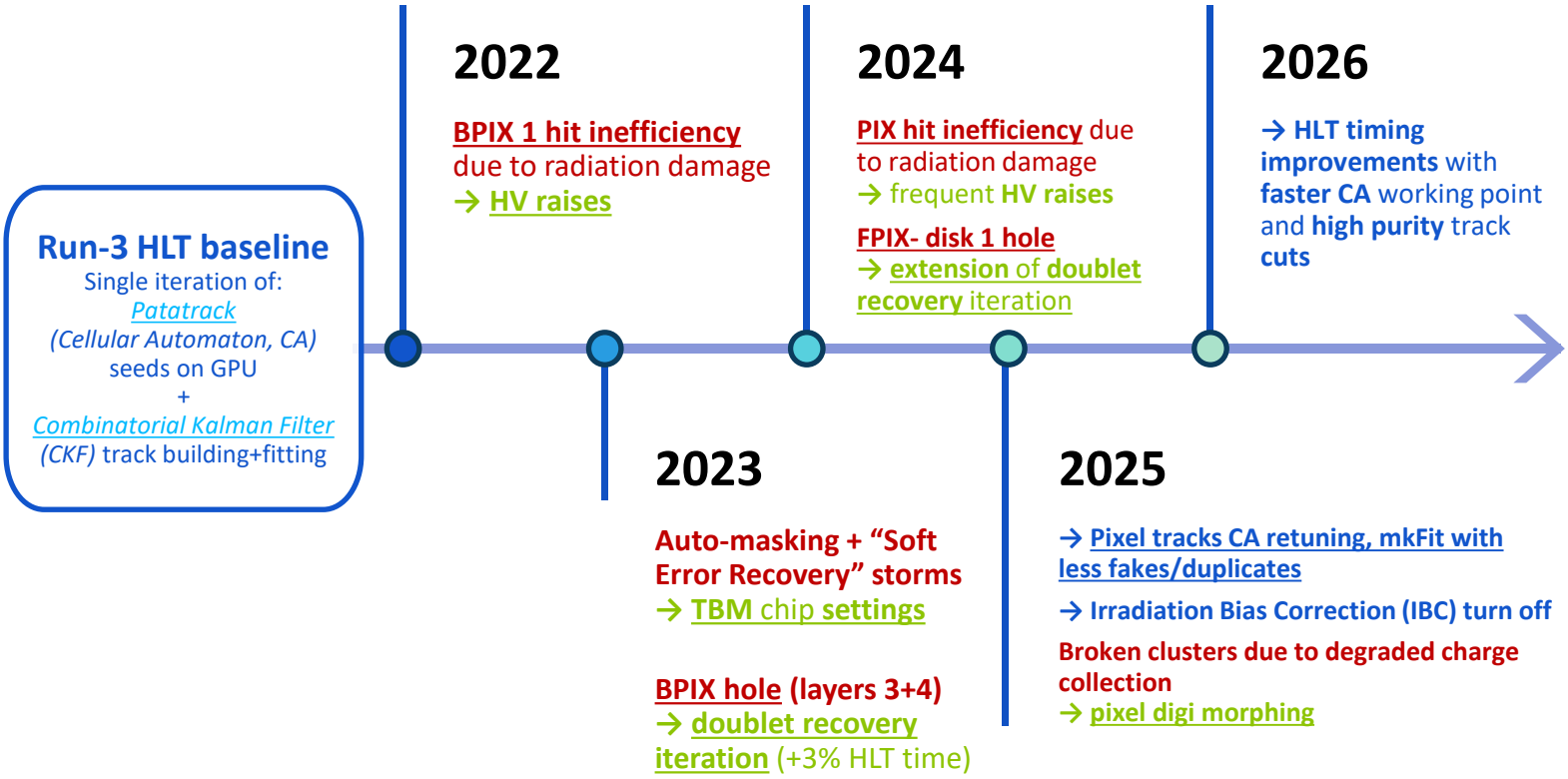
Run-3 and the Phase-2 Legacy baseline for HLT tracking

A look back at Run-3

Run-3 HLT baseline
Single iteration of:
Patatrack
(Cellular Automaton, CA)
seeds on GPU
+
Combinatorial Kalman Filter
(CKF) track building+fitting



A look back at Run-3



A look back at Run-3

Run-3 HLT baseline
Single iteration of:
Patatrack
(Cellular Automaton, CA)
seeds on GPU
+
Combinatorial Kalman Filter
(CKF) track building+fitting

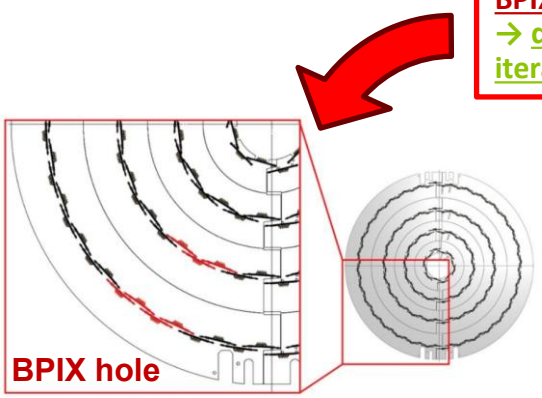
2022

BPIX 1 hit inefficiency due to radiation damage
→ **HV raises**

2023

Auto-masking + "Soft Error Recovery" storms
→ **TBM** chip **settings**

BPIX hole (layers 3+4)
→ **doublet recovery iteration** (+3% HLT time)



2024

PIX hit inefficiency due to radiation damage
→ frequent **HV raises**

FPIX- disk 1 hole
→ **extension** of **doublet recovery** iteration

2025

→ **Pixel tracks CA retuning, mkFit with less fakes/duplicates**

→ **Irradiation Bias Correction (IBC) turn off**

Broken clusters due to degraded charge collection
→ **pixel digi morphing**

2026

→ **HLT timing improvements** with **faster CA** working point and **high purity** track **cuts**

A look back at Run-3

Run-3 HLT baseline
Single iteration of:
Patatrack
(Cellular Automaton, CA)
seeds on GPU
+
Combinatorial Kalman Filter
(CKF) track building+fitting

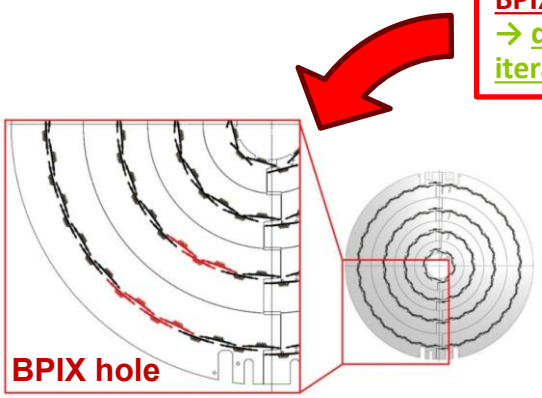
2022

BPIX 1 hit inefficiency due to radiation damage
→ **HV raises**

2023

Auto-masking + "Soft Error Recovery" storms
→ **TBM** chip **settings**

BPIX hole (layers 3+4)
→ **doublet recovery iteration** (+3% HLT time)



2024

PIX hit inefficiency due to radiation damage
→ frequent **HV raises**

FPIX- disk 1 hole
→ **extension** of **doublet recovery** iteration

2025

→ **Pixel tracks CA retuning, mkFit with less fakes/duplicates**

→ **Irradiation Bias Correction (IBC) turn off**

Broken clusters due to degraded charge collection
→ **pixel digi morphing**

2026

→ **HLT timing improvements** with **faster CA** working point and **high purity** track **cuts**

A look back at Run-3

Run-3 HLT baseline
Single iteration of:
Patatrack
(Cellular Automaton, CA)
seeds on GPU
+
Combinatorial Kalman Filter
(CKF) track building+fitting

2022

BPIX 1 hit inefficiency due to radiation damage
→ **HV raises**

2023

Auto-masking + "Soft Error Recovery" storms
→ **TBM** chip **settings**

BPIX hole (layers 3+4)
→ **doublet recovery iteration** (+3% HLT time)

2024

PIX hit inefficiency due to radiation damage
→ frequent **HV raises**

FPIX- disk 1 hole
→ **extension of doublet recovery** iteration

2025

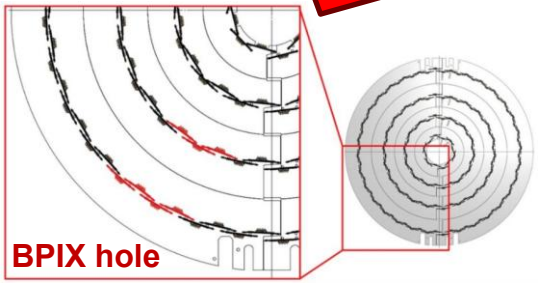
→ **Pixel tracks CA retuning, mkFit with less fakes/duplicates**

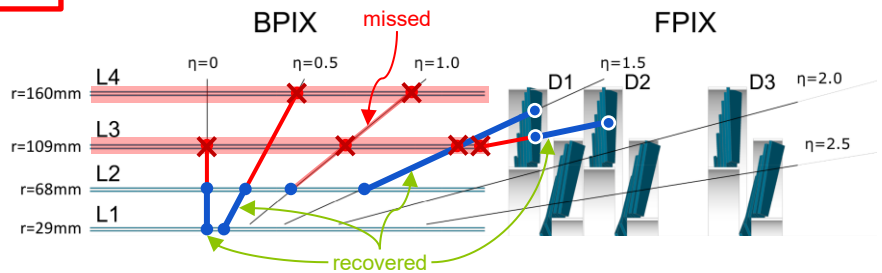
→ **Irradiation Bias Correction (IBC) turn off**

Broken clusters due to degraded charge collection
→ **pixel digi morphing**

2026

→ **HLT timing improvements** with **faster CA** working point and **high purity** track **cuts**





doublet recovery of missed triplets

A look back at Run-3

Run-3 HLT baseline
Single iteration of:
Patatrack
(Cellular Automaton, CA)
seeds on GPU
+
Combinatorial Kalman Filter
(CKF) track building+fitting

2022

BPIX 1 hit inefficiency due to radiation damage
→ **HV raises**

2023

Auto-masking + "Soft Error Recovery" storms
→ **TBM chip settings**

BPIX hole (layers 3+4)
→ **doublet recovery iteration** (+3% HLT time)

2024

PIX hit inefficiency due to radiation damage
→ frequent **HV raises**

FPIX- disk 1 hole
→ **extension of doublet recovery** iteration

2025

→ **Pixel tracks CA retuning, mkFit with less fakes/duplicates**

→ **Irradiation Bias Correction (IBC) turn off**

Broken clusters due to degraded charge collection
→ **pixel digi morphing**

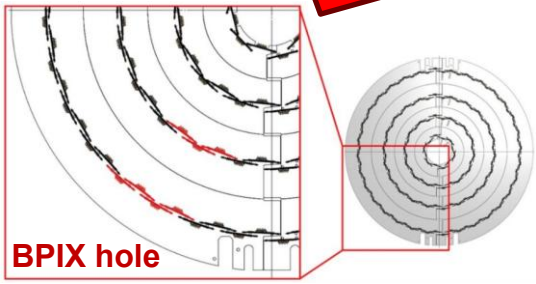
2026

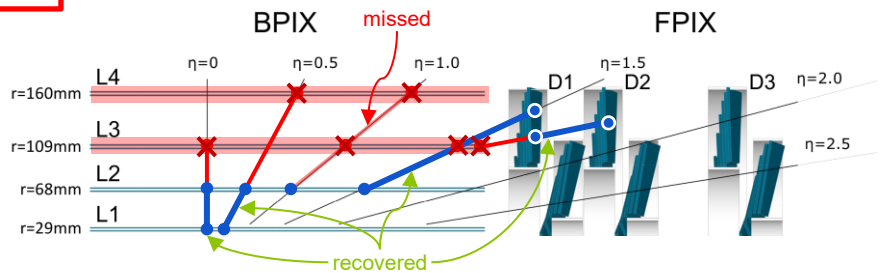
→ **HLT timing improvements** with **faster CA** working point and **high purity** track **cuts**

Takeaway for Phase-2

Strong interplay:
HLT physics/timing performance
↕
Tracker conditions

→ Build tracking chain with **failure scenarios** in mind, e.g. introducing **redundancy**

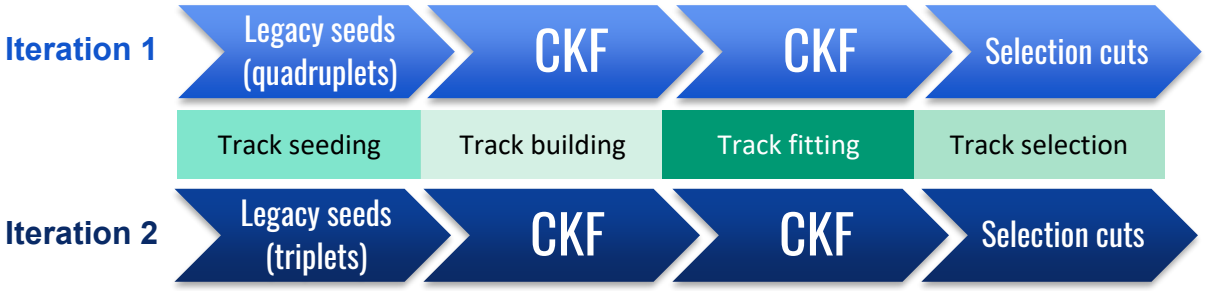




doublet recovery of missed triplets

For more [CMS-TDR-022]

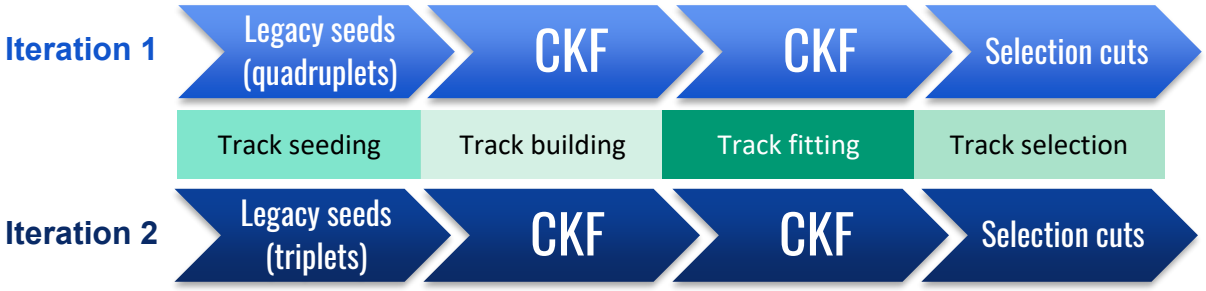
HLT Legacy baseline for Phase-2

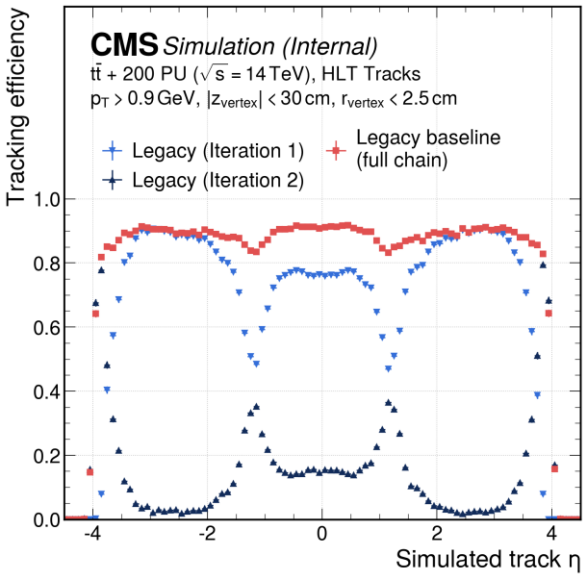


- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations

For more [CMS-TDR-022]

HLT Legacy baseline for Phase-2

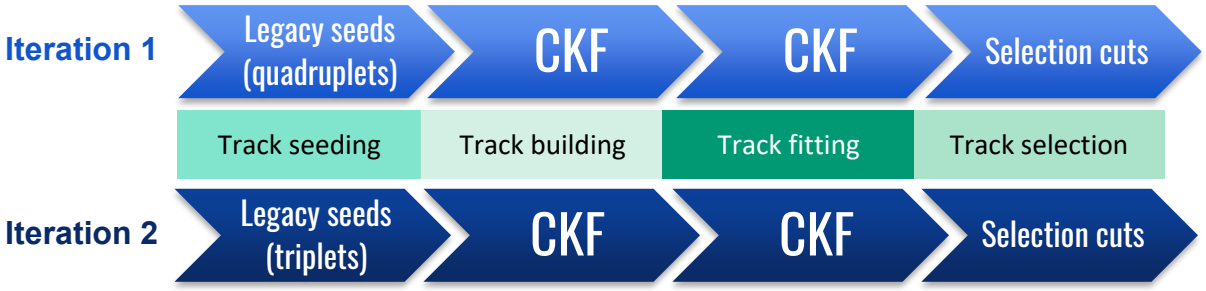




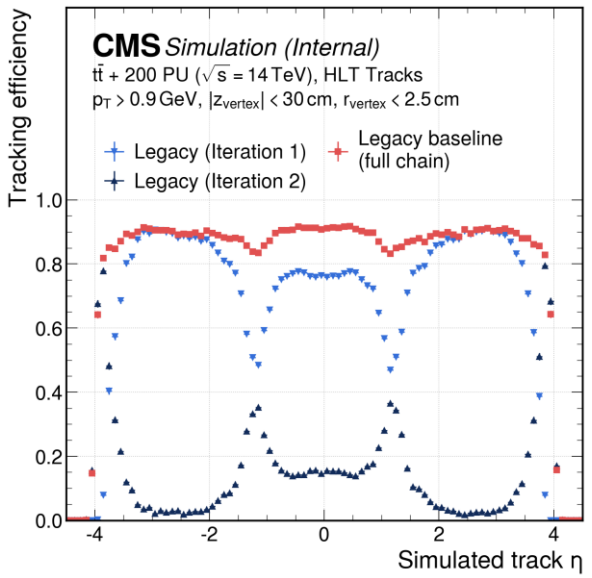
- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**

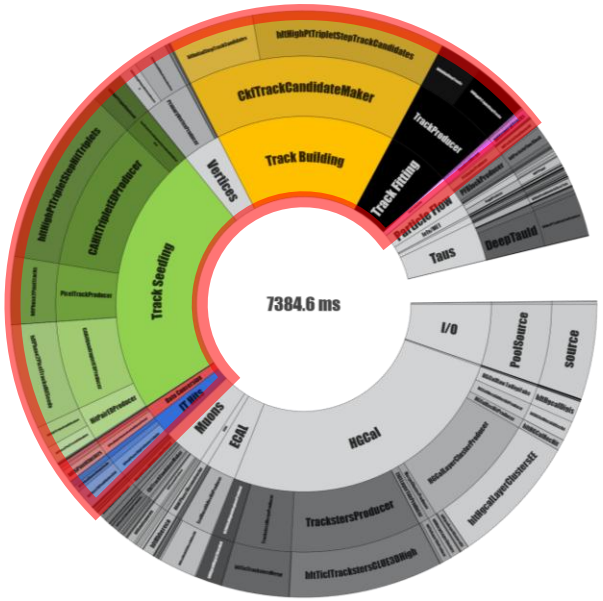
For more [CMS-TDR-022]

HLT Legacy baseline for Phase-2



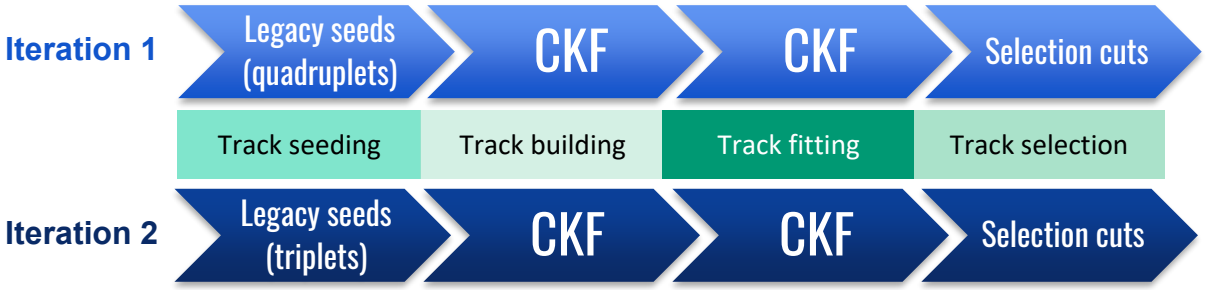
- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**
  - **Timing** is arguably still far from what we need



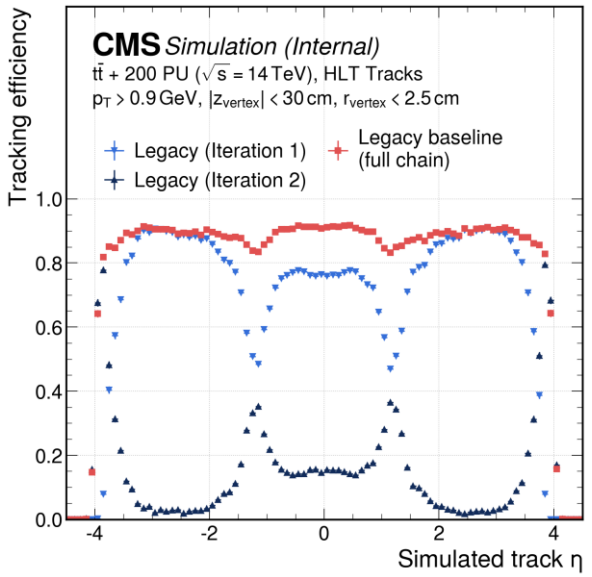


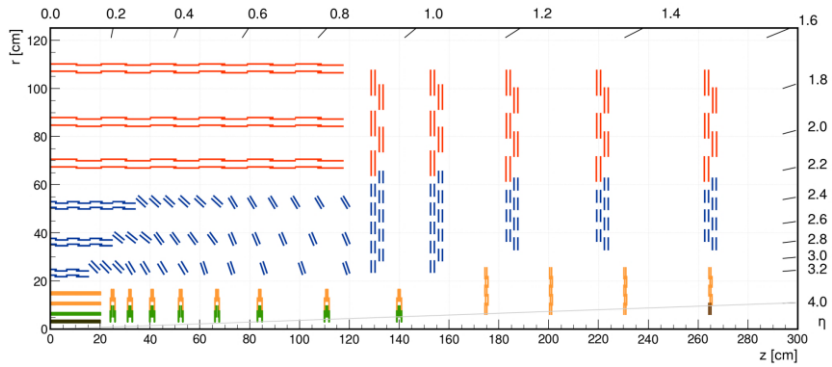
For more [CMS-TDR-022]

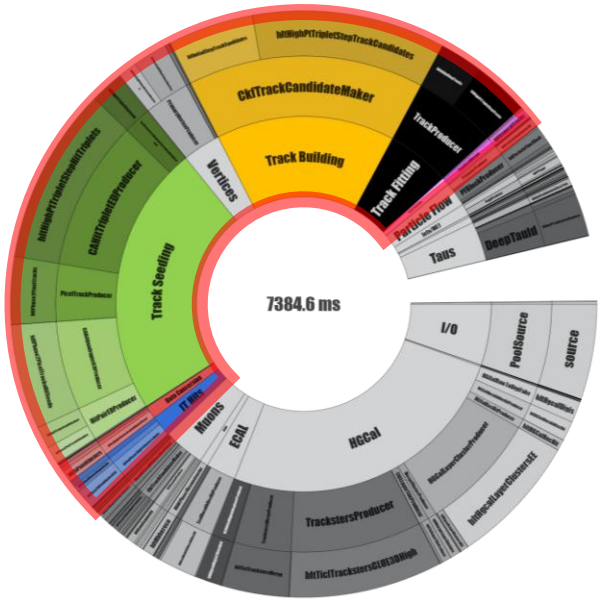
HLT Legacy baseline for Phase-2



- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**
  - **Timing** is arguably still far from what we need
  - Does not yet fully exploit the new tracker geometry

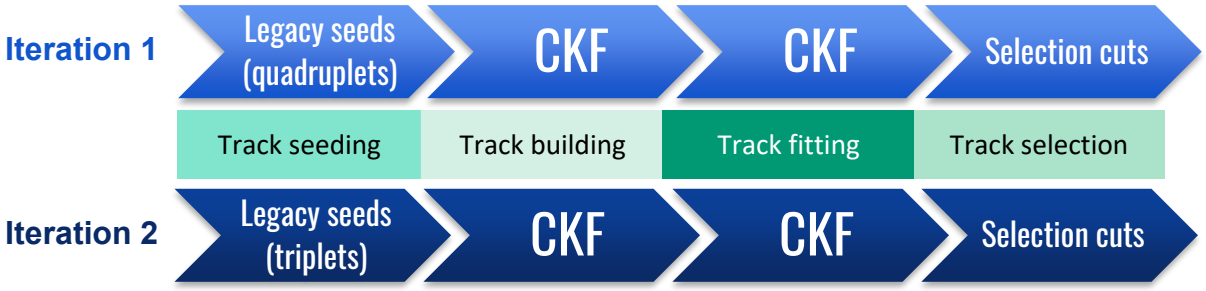


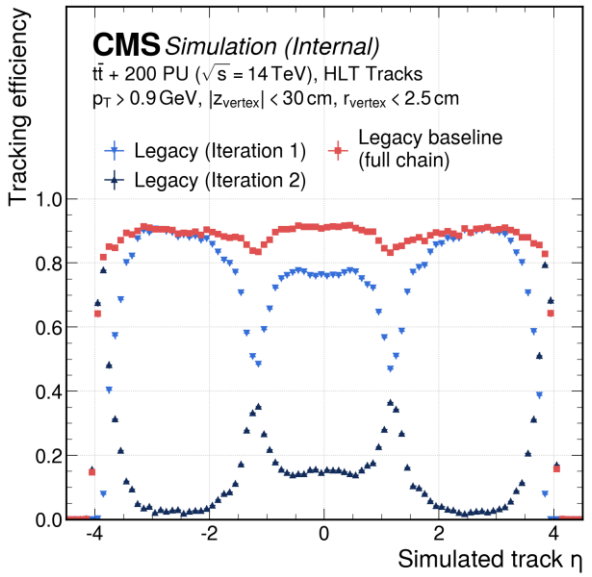




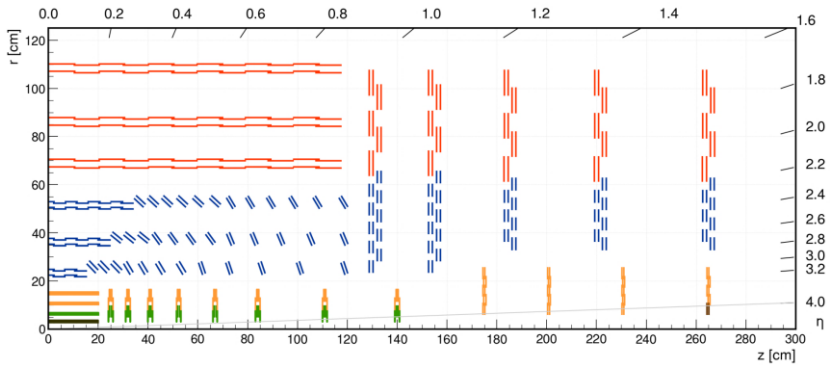
For more [CMS-TDR-022]

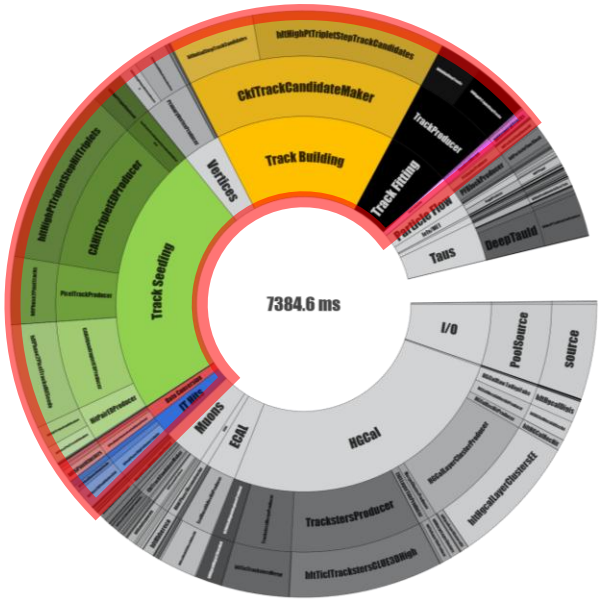
HLT Legacy baseline for Phase-2





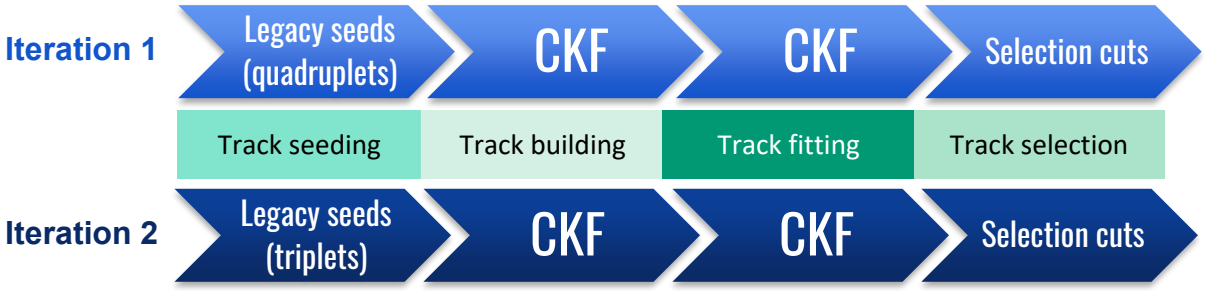
- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**
  - **Timing** is arguably still far from what we need
  - Does not yet fully exploit the new tracker geometry
    - Indirectly yes, local reconstruction, phase-space coverage



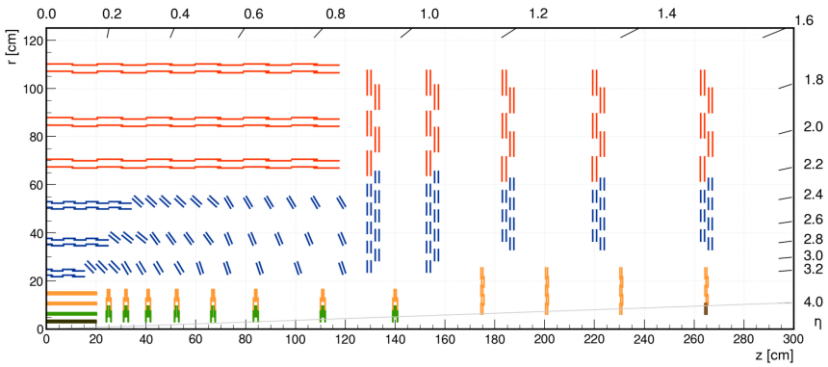


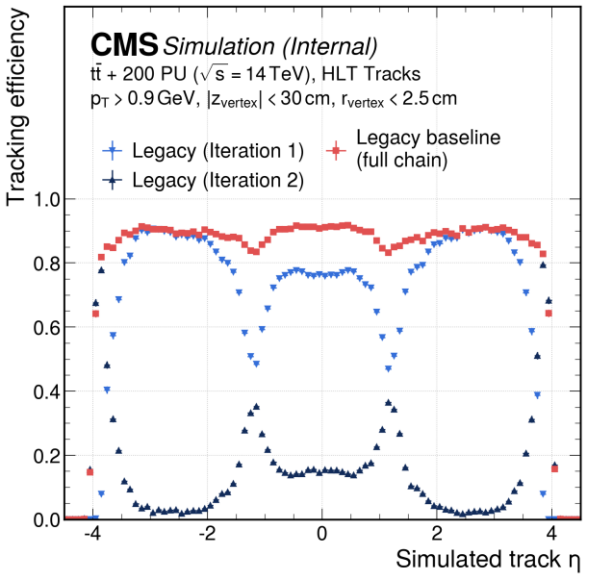
For more [CMS-TDR-022]

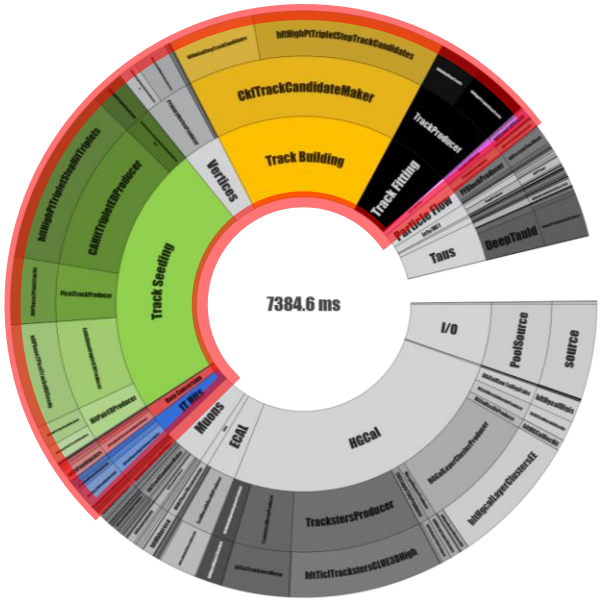
HLT Legacy baseline for Phase-2



- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**
  - **Timing** is arguably still far from what we need
  - Does not yet fully exploit the new tracker geometry
    - Indirectly yes, local reconstruction, phase-space coverage
    - BUT not e.g. p_T-modules in the OT, or large redundancy in IT endcap

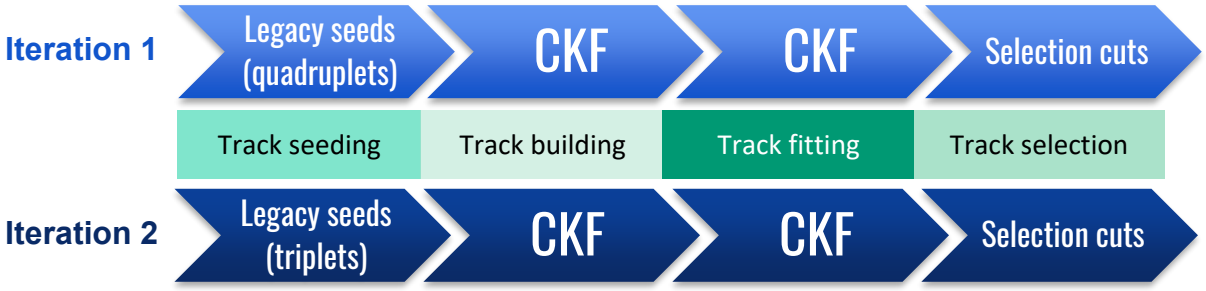


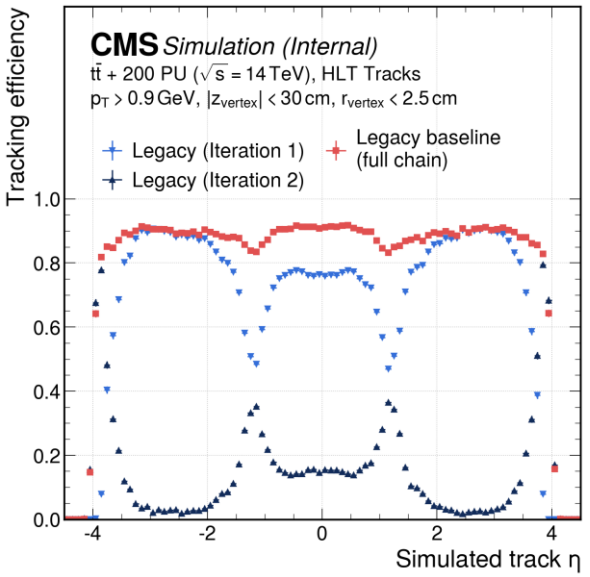




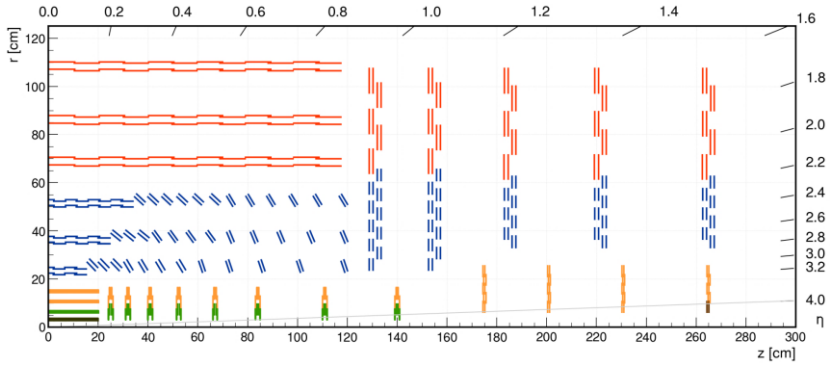
For more [CMS-TDR-022]

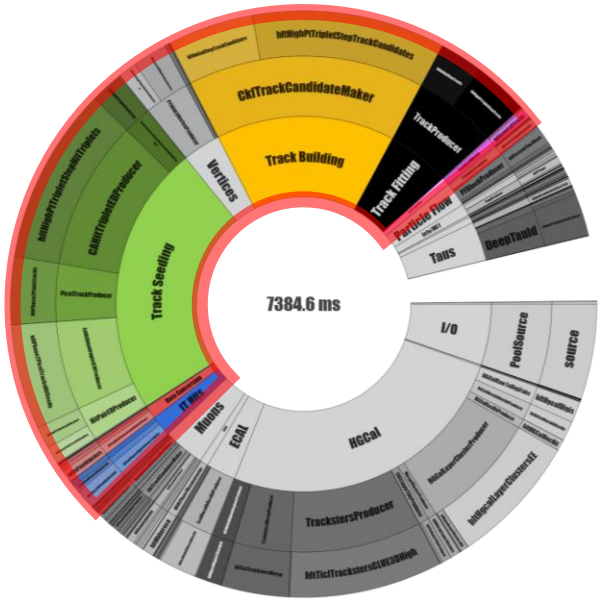
HLT Legacy baseline for Phase-2





- **Initial "Legacy" baseline based on Run-2 menu:**
  - Iterative tracking approach as offline (see Breno's talk)
    - BUT faster → use only 2 iterations
  - Good reference baseline to have with solid **physics performance**
  - **Timing** is arguably still far from what we need
  - Does not yet fully exploit the new tracker geometry
    - Indirectly yes, local reconstruction, phase-space coverage
    - BUT not e.g. p_T-modules in the OT, or large redundancy in IT endcap
- **Need for new tracking algorithms and tracking chain for HLT!**





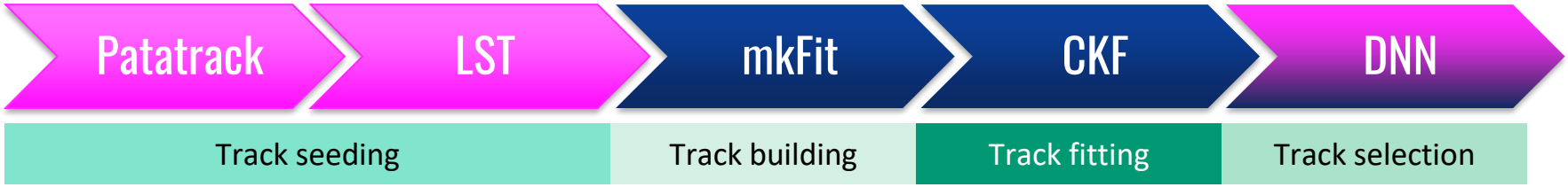
Tracking the Present

The new Phase-2 baseline for HLT tracking

(New) current baseline

GPU | Alpaka

CPU



Physics performance
→ retain or even improve tracking output for optimal event selection + scouting

Computational performance
→ reduce timing for being able to run (more)

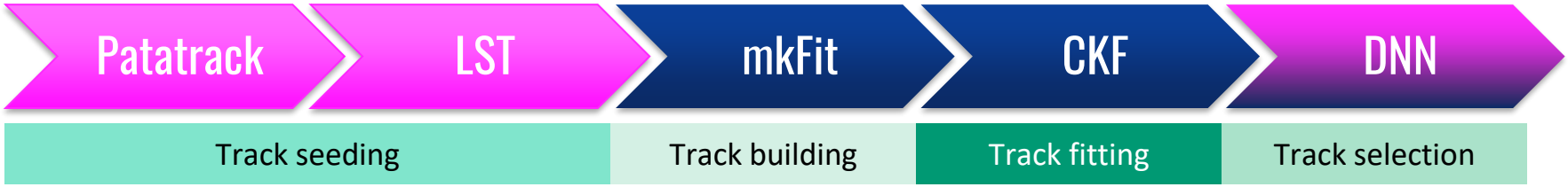
→ **How to achieve this?**
- Exploit upgraded tracker
- Exploit multi-core CPU architectures
  - Using parallelized and vectorized algorithms
- Exploit heterogeneous computing models (CPUs + GPUs)
  - Using algorithms that are suitable for GPU
- Enhance / extend usage of ML techniques
- Eventually, combine all of the above!

(New) current baseline

GPU | Alpaka

CPU





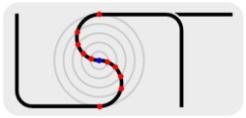
Physics performance
→ retain or even improve tracking output for optimal event selection + scouting

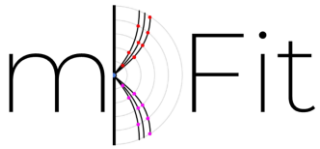
Computational performance
→ reduce timing for being able to run (more)

→ **How to achieve this?**
- Exploit upgraded tracker
- Exploit multi-core CPU architectures
  - Using parallelized and vectorized algorithms
- Exploit heterogeneous computing models (CPUs + GPUs)
  - Using algorithms that are suitable for GPU
- Enhance / extend usage of ML techniques
- Eventually, combine all of the above!

alpaka







- **Current state-of-the-art tracking:**
  - New HLT baseline since March 2026
  - Builds on three main algorithms:
    1. Patatrack for track seeding (GPU-only fitted tracks)
    2. Line Segment Tracking (LST) for track seeding (GPU)
    3. mkFit for track building (and fitting)
  - **All of them are being improved further with recent and ongoing developments**

Patatrack



More details here





Patatrack → LST → mkFit → CKF → DNN

GPU | Alpaka: RecHits → Doublets → Triplets → N-tuplets → BL Fit → DNN

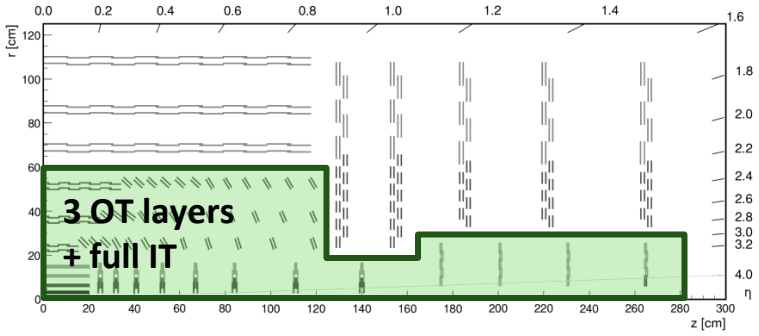
Input

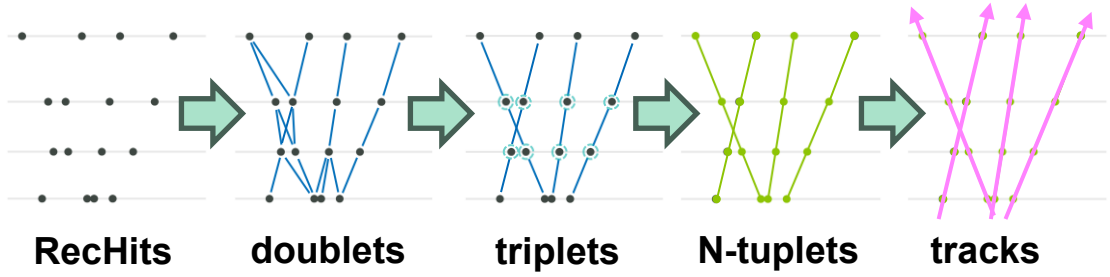
- IT RecHits + OT Pixel Hits (TrackingRecHitSoA)

Output

- Pixel Tracks (TracksSoA)

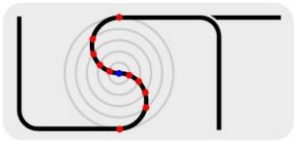
- **Patatrack = parallel, hardware-agnostic (pixel) track reconstruction**
  - Already in use by CMS during Run 3, for track seeding at HLT
  - Main steps following the (pixel) tracker local reconstruction:
    1. Link RecHits → N-tuplets using Cellular Automaton (CA) algorithm
    2. Broken Line fit (BL) to estimate track parameters
    3. Clean track collection for duplicates + fake rejection via DNN
- **For HL-LHC, Patatrack algorithm has been extended to OT**
  - Using information from IT + pixel sensors of OT layers
  - More redundancies thanks to more hits, ability to skip layers
- **Work is ongoing on further extensions / improvements, towards:**
  - Enabling iterative CA - rather than a single, global iteration (more in Breno's talk)
  - Using vector hits in OT - rather than using only pixel sensors (more later on ITACA)
  - Improvements to material/magnetic field maps in Broken Line fit soon (currently single value-based see also Nazar's talk)





Line Segment Tracking (LST)





many more details here

Patatrack | **LST** | mkFit | CKF | DNN

GPU | Alpaka

MD → Line Segments → T3 → T4 / T5 → pT3 / pT5 → track cands.

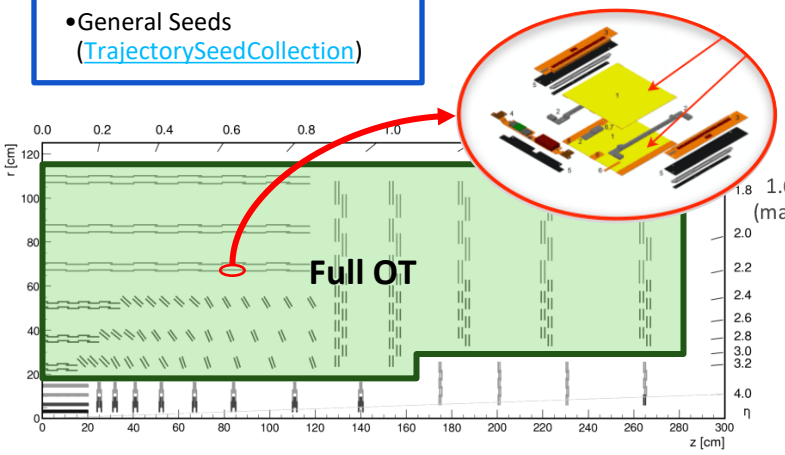
Input

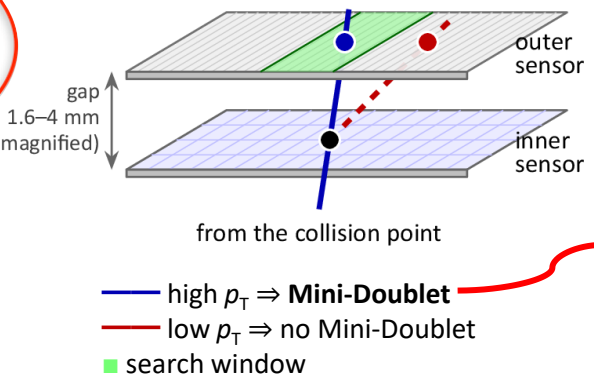
- LST input (LSTInputSoA)
  - → OT RecHits (Phase2TrackerRecHit1D)
  - → Pixel Seeds from Patatrack (TrajectorySeedCollection)

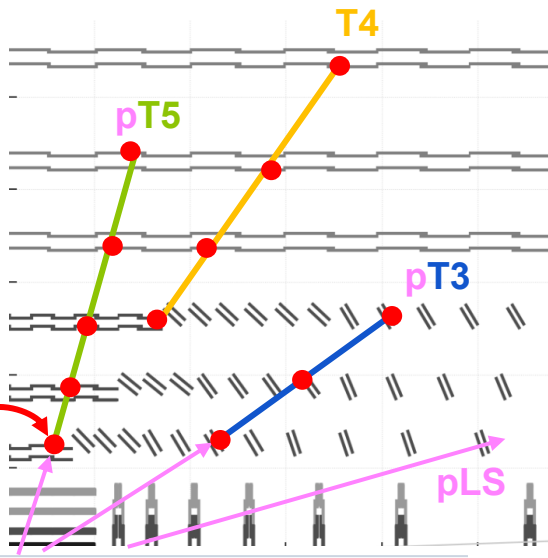
Output

- General Seeds (TrajectorySeedCollection)

- **LST** ≔ **Heterogeneous, parallel track finding algorithm for the OT of Phase-2**
  - Combine OT hits to "Mini-Doublets" (**MD**, custom GPU-built stubs for $p_T > 0.8$ GeV since 2024)
    - = hit on lower + hit on upper sensor of the same PS/2S module
    - search window considers minimum track-p_T, multiple scattering, vertex-offset and tilt allowances
  - combine compatible mini-doublets of adjacent layers to: Line Segments (**LS**) → Triplets (**T3**) → Quadruplets / Quintuplets (**T4 / T5**)
  - connect those OT-only objects to pixel seeds (**pLS**)
  - Final candidates: pT3, pT4, pT5, T4, T5, single pLS
- **LST efficiently enables displaced tracking (T4 & T5)**







mkFit



More details here

Patatrack | LST | mkFit | CKF | DNN

CPU | TBB + SIMD — Hits | Seeds | Building | Backward | Duplicates | Output

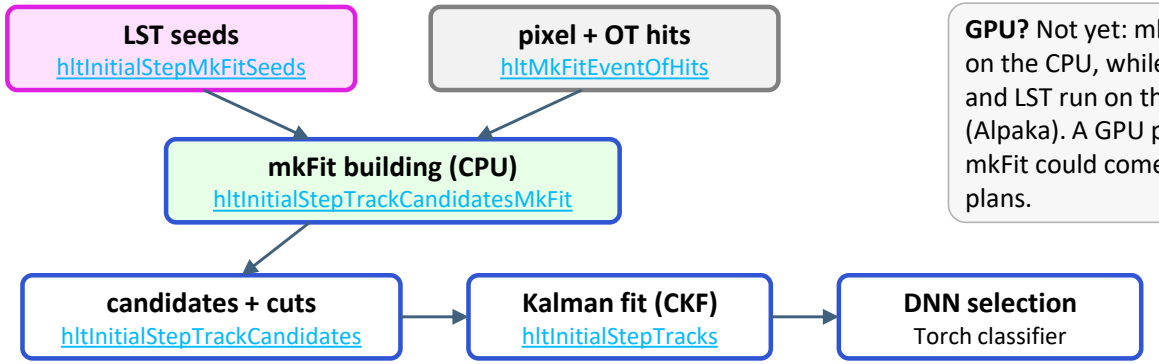
- **mkFit** ≔ **Kalman-filter combinatorial track finding** (and fitting) on the CPU
  - was already used in Run-3 HLT and offline for >90% of tracks with $p_T > 0.5$ GeV
  - **SIMD** (single instruction, multiple data): Matriplex processes 16 candidates at once (AVX-512)
  - **TBB** (Intel Threading Building Blocks, as in CMSSW): threads over η regions and seeds
  - each LST seed starts at the layer of its last hit: extended **outwards**, then **inwards**
  - **full tracker**: pixel + OT, each stacked-module sensor is its own layer

Input

- Seeds = LST track candidates (hltInitialStepTrajectorySeedsLST)
- Pixel + OT hits in mkFit format (hltMkFitEventOfHits)
- Geometry + iteration config (hltESPMkFit): 60 layers, mkfit-phase2-lstStep.json

Output

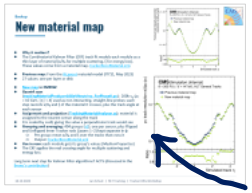
- Track candidates (hltInitialStepTrackCandidates)
  → Kalman track fit (CKF)
  → DNN selection



GPU? Not yet: mkFit runs on the CPU, while Patatrack and LST run on the GPU (Alpaka). A GPU port of mkFit could come in the plans.

cms-sw/cmssw@f6063aa HLTInitialStepSequence, RecoTracker/MkFit, MkFitCore

New material map for CKF

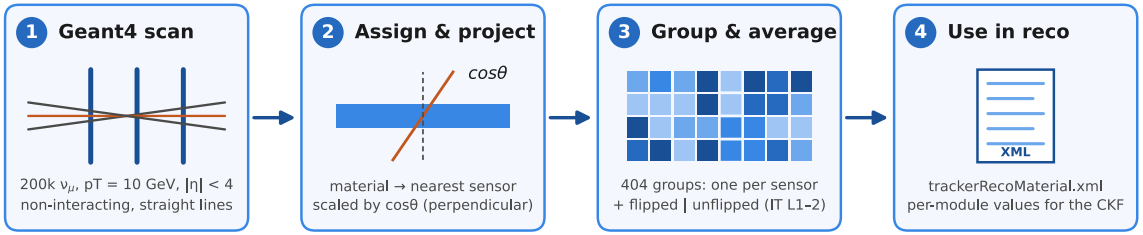


More details here

Why it matters
The CKF treats each module as a thin scatterer: x/X_0 drives multiple scattering and ξ drives energy loss. Both come from a material map (trackerRecoMaterial.xml).

Before: tkLayout model (IT711, 2023), 27 values, one per layer or disc

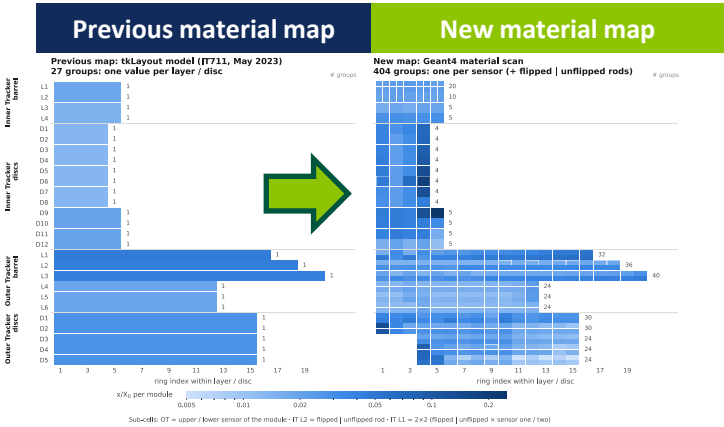
Now: Geant4-derived map, 404 values, one per sensor.
Procedure for derivation and usage:

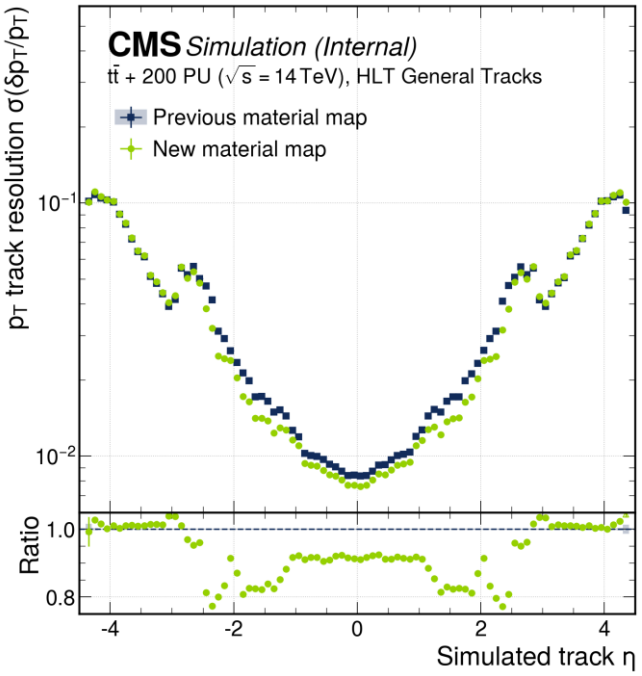


Per-sensor mean x/X_0 and ξ replace one value per layer / disc

In reco: each module gets its group's values, and the CKF applies the real crossing angle.
Try to share maps across various tracking/fitting algorithms.

Next step: ACTS for better passive material description (see Nazar's talk, Breno's talk)

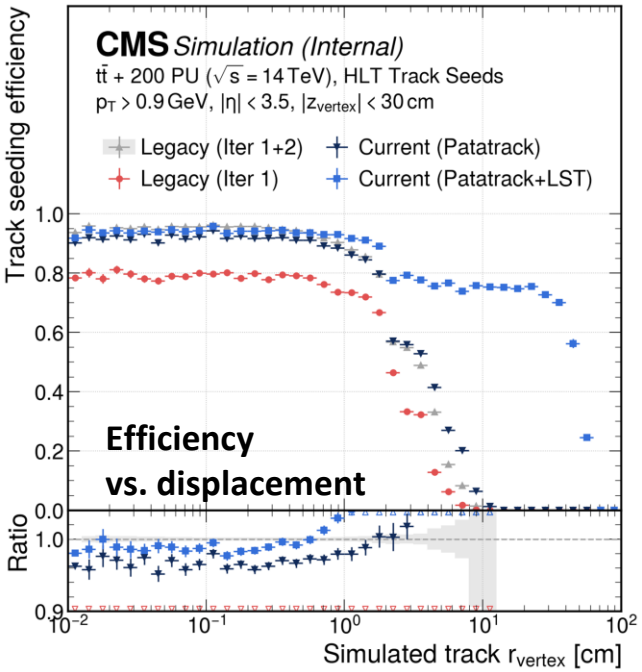


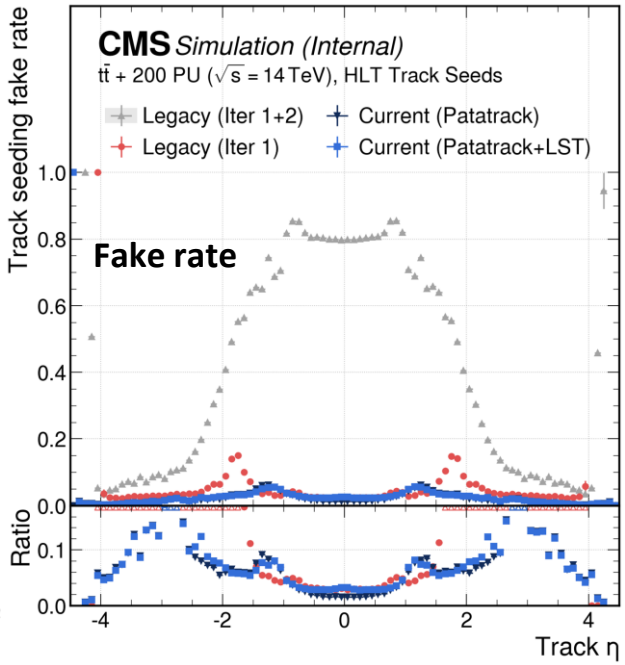


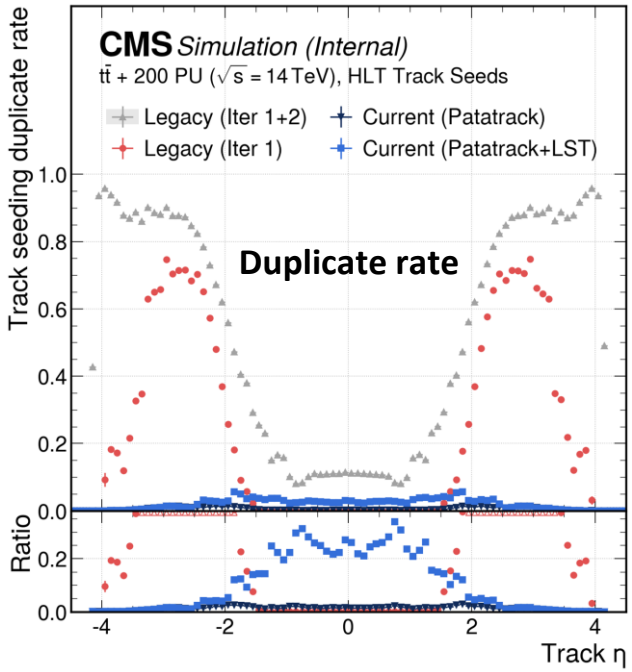
*Performance of release 20_1_0_pre4
+ latest developments in LST & mkFit
(PRs #52011, #52015, #52129)

Physics performance - Seeding

- **Large improvements with Patatrack pixel tracking enable moving to a single-iteration approach**
- **LST adds displaced+prompt seeding on top, extending physics reach to far displaced phase-spaces**
- **Great reduction of fakes at the seeding level thanks to combination of**
  - better utilization of the given Tracker
  - longer track seeds
  - DNN-based fake+duplicate rejections at various stages in Patatrack & LST





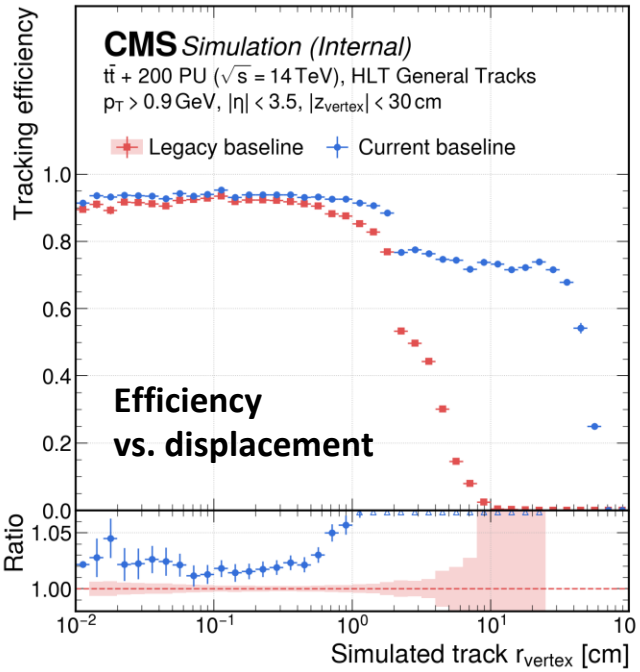


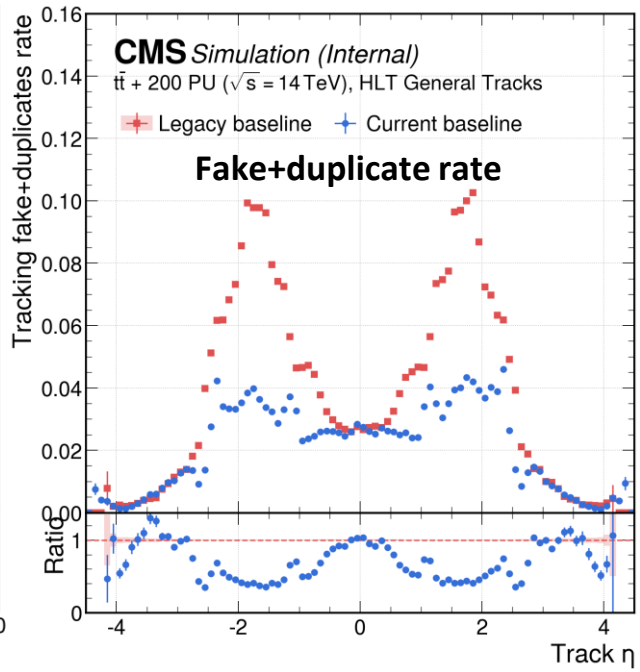
[DP-2026-026] for more

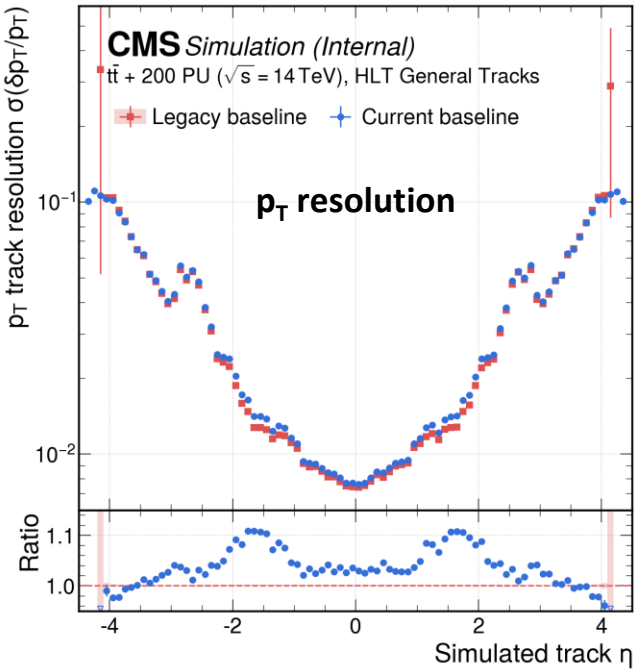
*Performance of release 20_1_0_pre4
+ latest developments in LST & mkFit
(PRs #52011, #52015, #52129)

Physics performance

- **Current baseline shows great physics performance with**
  - Higher efficiency than Legacy + extension of physics reach to very displaced tracks
  - Lower fake & duplicate rate than Legacy
  - Comparable quality of track parameter estimates (note that the curves do not contain the same set of tracks → additional high-displacement tracks







[DP-2026-026] for more

Timing performance

Significant reduction in HLT tracking time:
4.9 s → 1.1 s for track reconstruction per evt

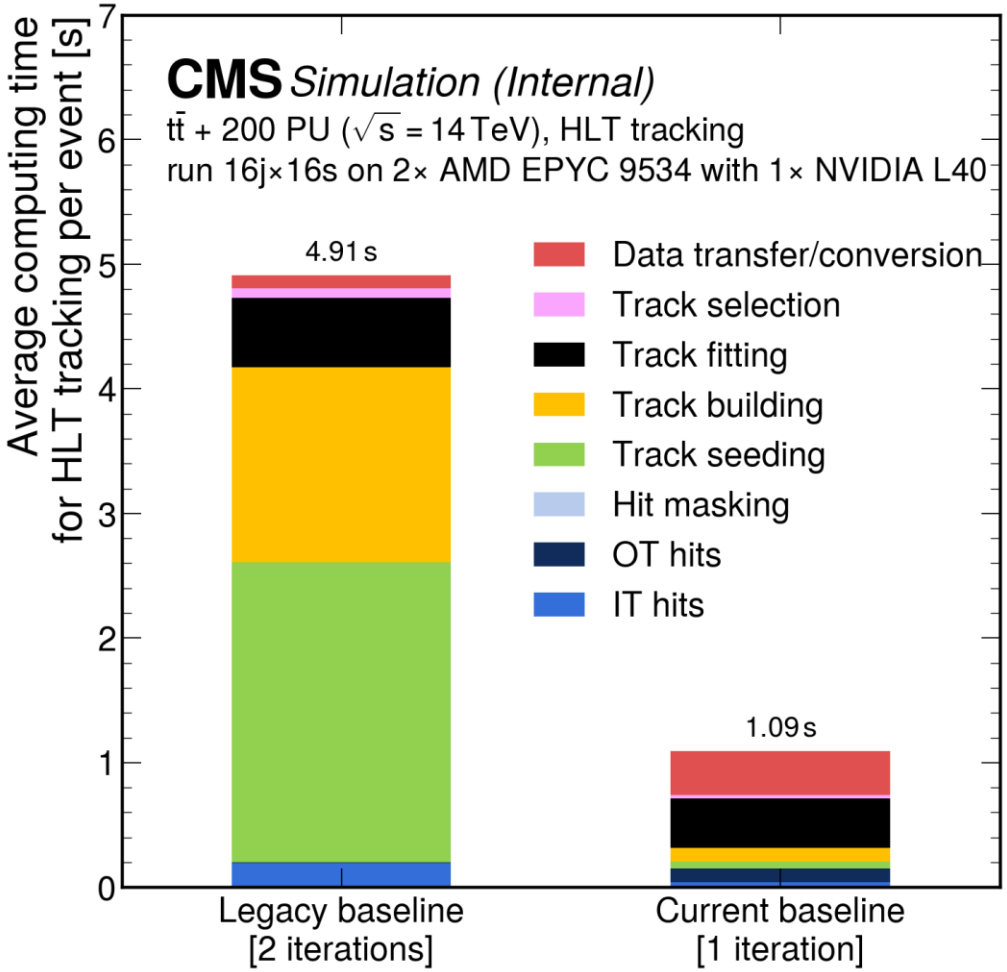
Thanks to various improvements:

Cutting the second iteration entirely
- Possible thanks huge improvements in initial seeding by ***Patatrack*** compared to Legacy (Iteration 1)

Offloading to GPUs
- Huge speedup in seeding (***Patatrack+LST***)
- cost of some extra conversion time *(work ongoing)*

Much cleaner seeding (*Patatrack+LST*)
- Less bad tracks (fakes/duplicates) that need to be built+fitted

Speed-up of track building
- Building with ***mkFit*** instead of CKF
- cost of some extra conversion time *(work ongoing)*



Trailing the Future

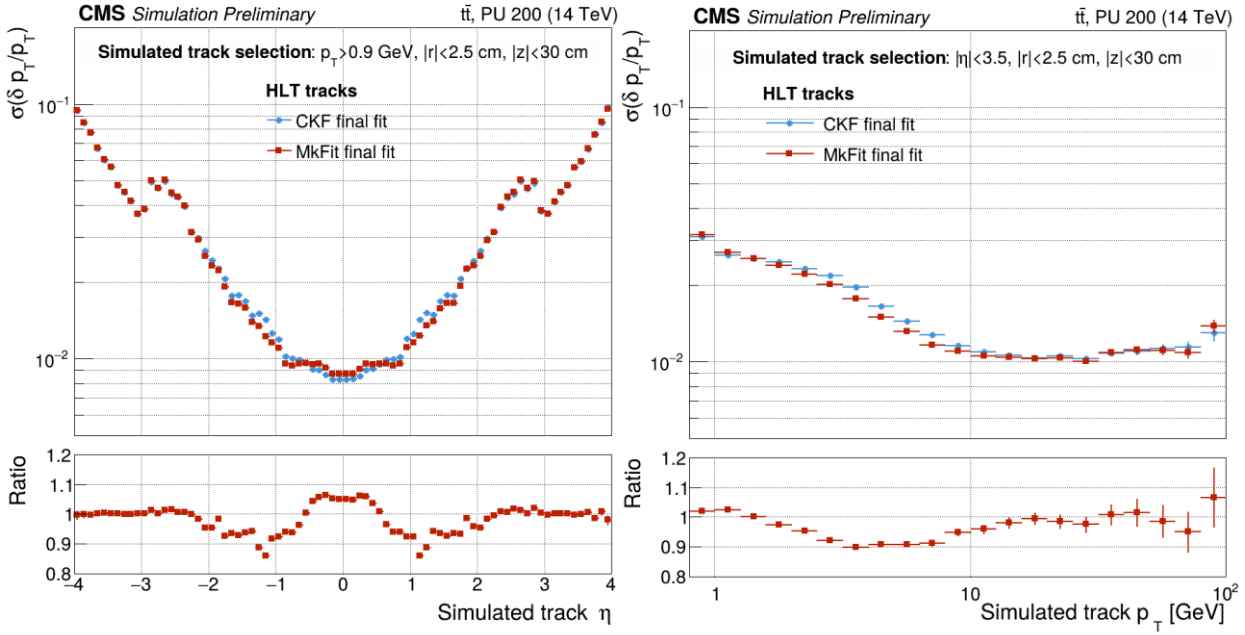
Ongoing developments for Phase-2 HLT tracking

mkFit fit

For more [CMS-DP2026-031]

- **Currently mkFit is used for track building**
- **mkFit fit could serve as a replacement of CKF track fitting**

- In the new baseline, track fitting is about as costly (time-wise) as track building
- **Enabling mkFit track fitting can further improve time**



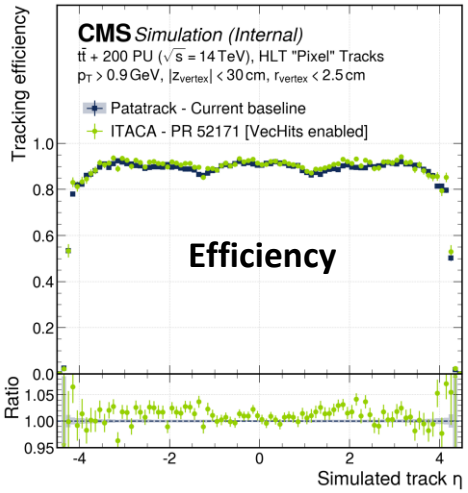
	Track building	Conversion (track candidate)	Track fitting	Conversion (track)	Fitting + Conversion	Full tracking (with GPUs)	Full tracking (with CPUs)
CKF final fit	92 ms	18 ms	300 ms	-	318 ms	708 ms	1687 ms
MkFit final fit		-	50 ms	42 ms	92 ms	484 ms	1456 ms
Reduction					71%	32%	14%

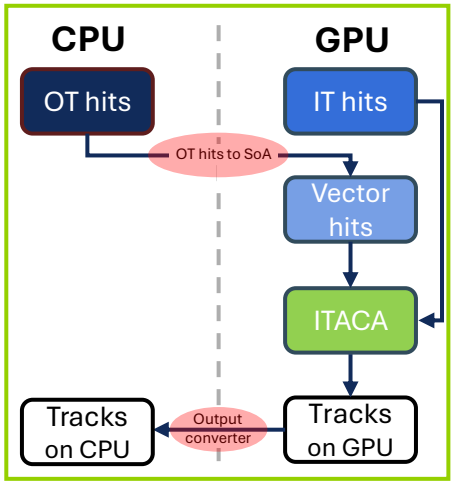
Trailing the Future

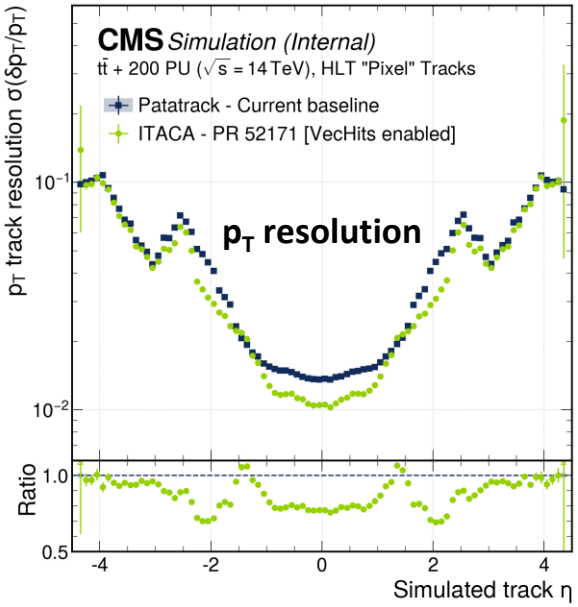
ITACA

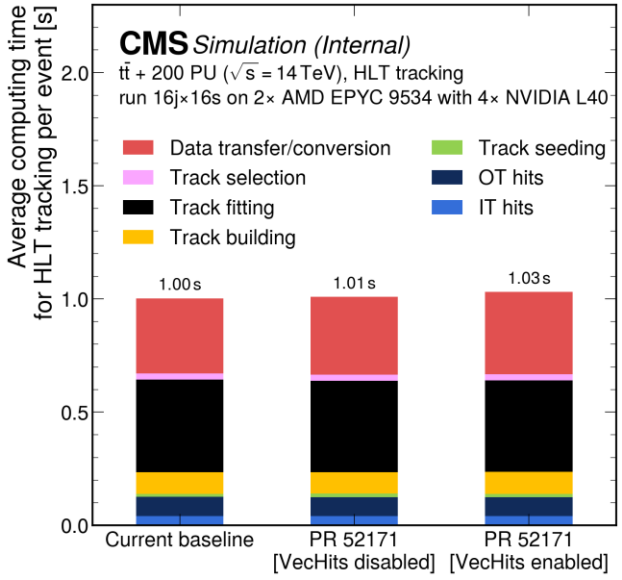
*Performance of IB CMSSW_20_1_X_2026-10-04-0000
+ latest development in Patatrack/ITACA
(PR #52171 w and w/o procModifier phase2CAVectorHits)

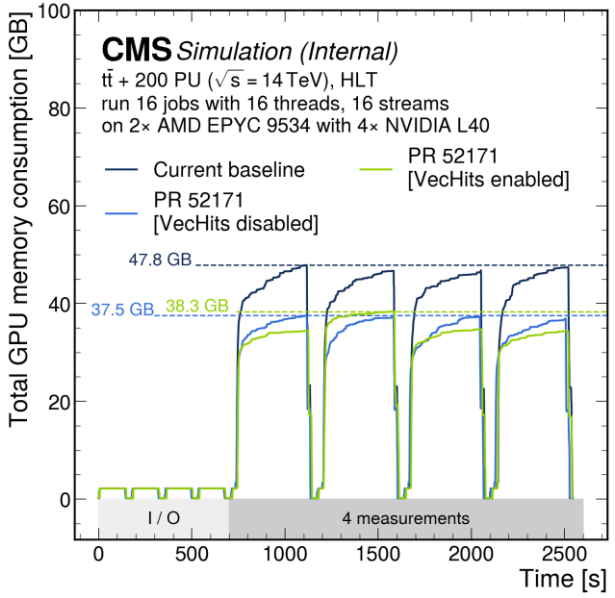
- **ITACA is an evolution of Patatrack pixel tracking**
- **Key changes with respect to current Patatrack in release:**
  - Utilization of the **full tracker** instead of IT + first 3 OT layers
  - Builds and uses **vector-hits** in the OT (see also Marco's talk, original work by Erica)
    - Overlap with LST's internal mini-doublets
- Like Patatrack: **fully heterogeneous chain** on GPU
- **Performance:**
  - Better p_T resolution in track seeding
  - Timing similar to baseline
  - Memory improved to baseline
    - w and w/o VecHits similar











Ready for failure scenarios?

- **Learn from Run-3 and try to include redundancies early on**

Ready for failure scenarios?

- **Learn from Run-3 and try to include redundancies early on**
- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:

Ready for failure scenarios?

- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)

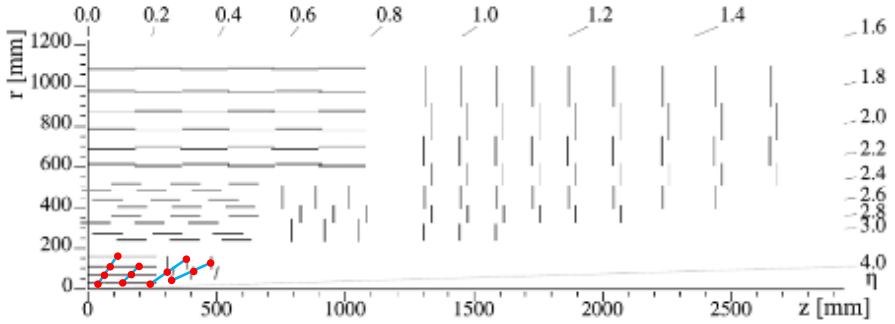
Ready for failure scenarios?

- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)

Start of Run-3

Example: Patatrack Tracks (Seeding)



Ready for failure scenarios?

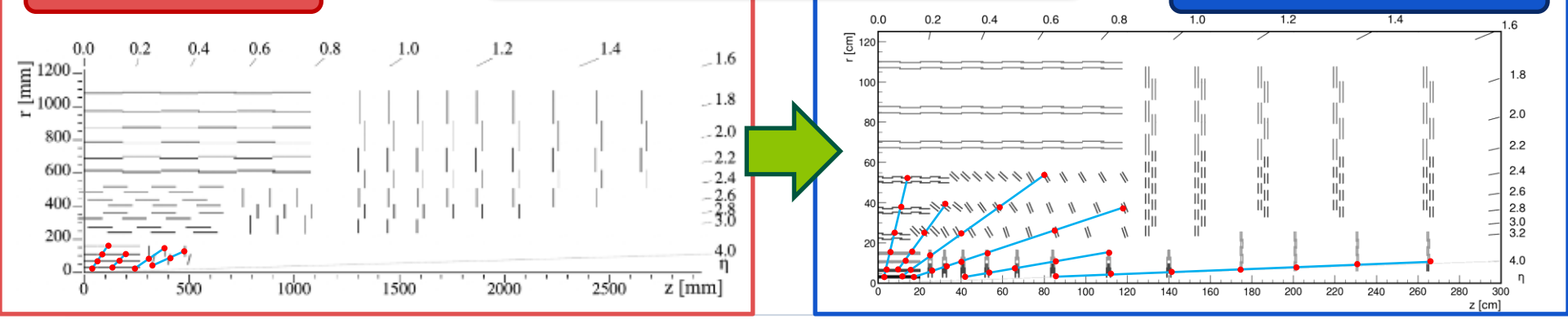
- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)

Start of Run-3

Example: Patatrack Tracks (Seeding)

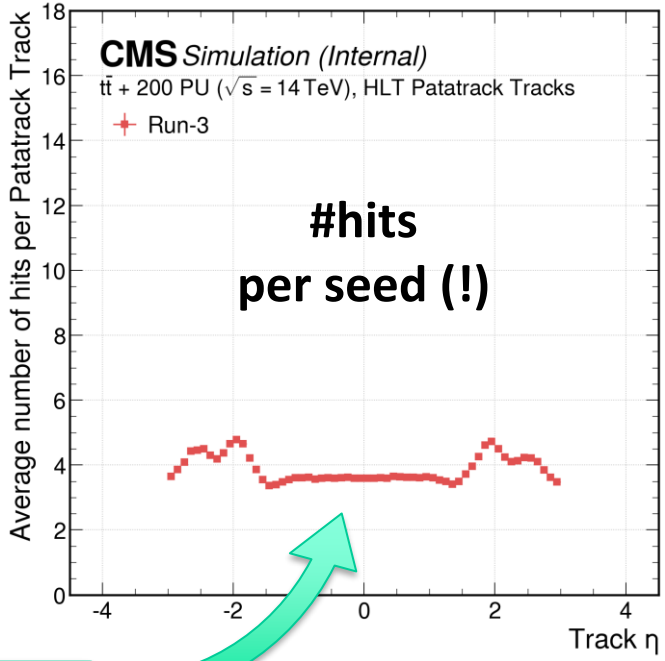
Current Phase-2 baseline



Ready for failure scenarios?

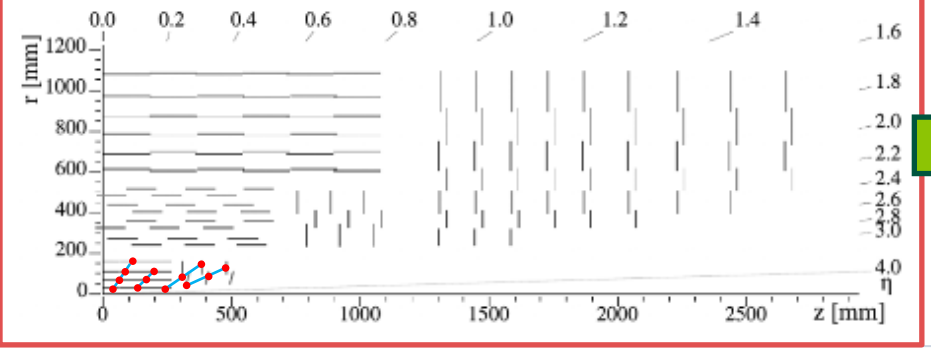
- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)

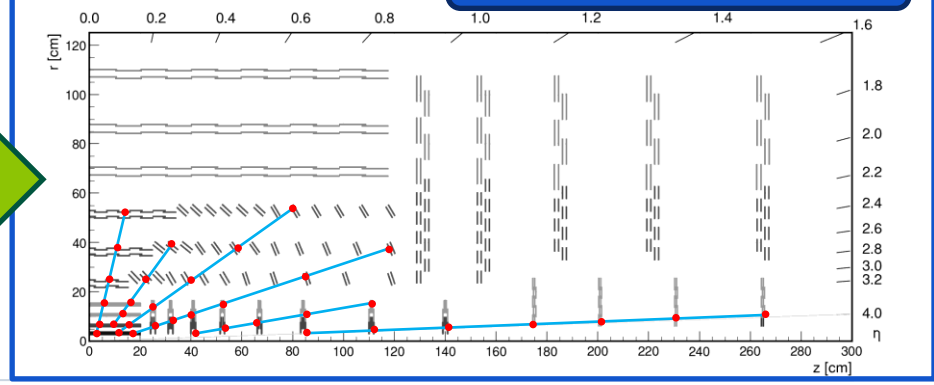


Example: Patatrack Tracks (Seeding)

Start of Run-3



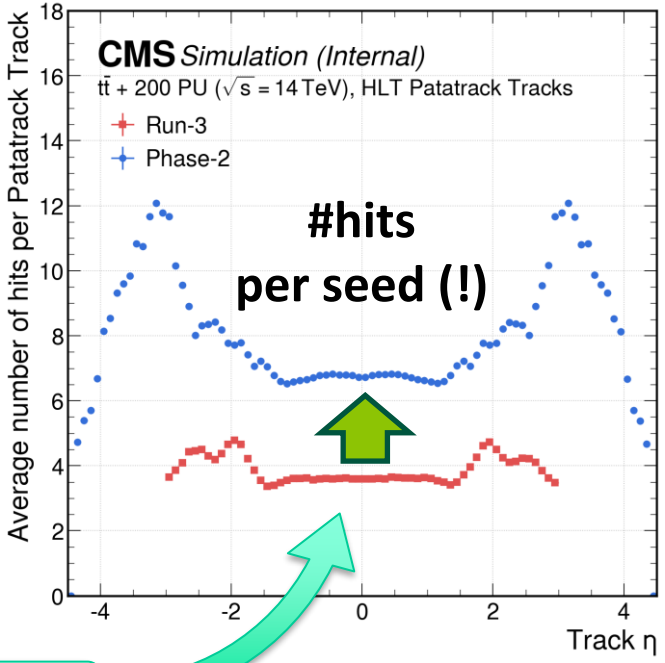
Current Phase-2 baseline



Ready for failure scenarios?

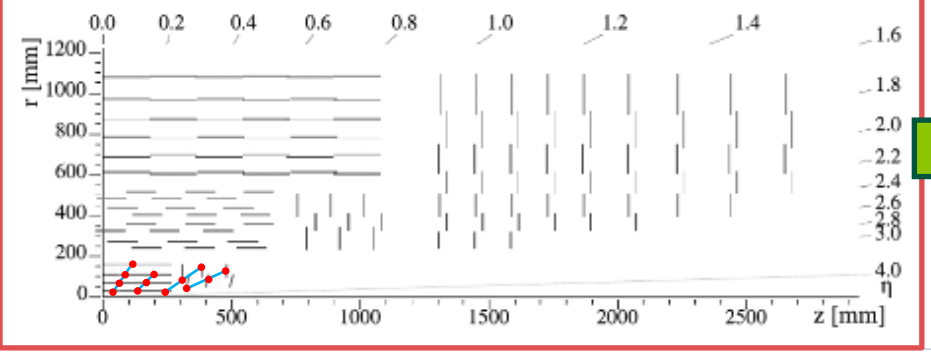
- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)

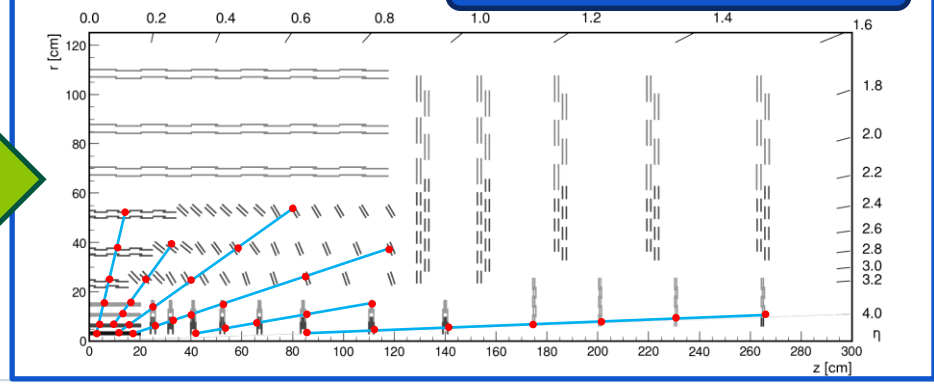


Example: Patatrack Tracks (Seeding)

Start of Run-3



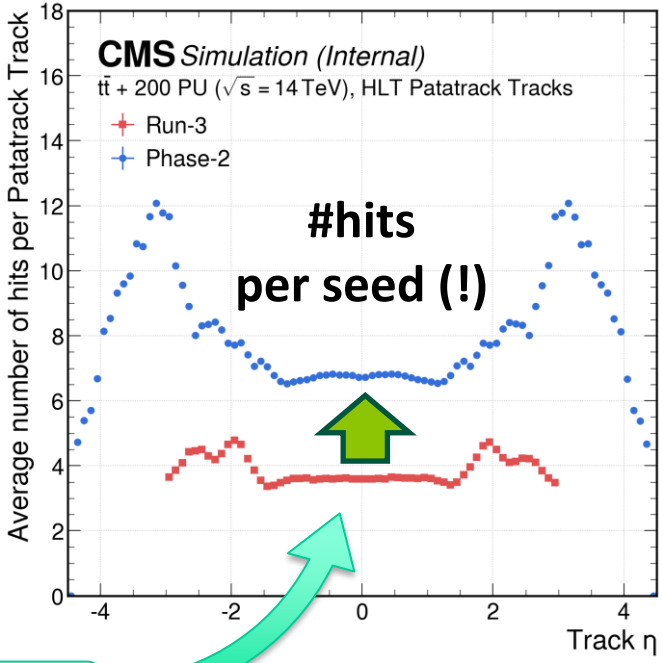
Current Phase-2 baseline



Ready for failure scenarios?

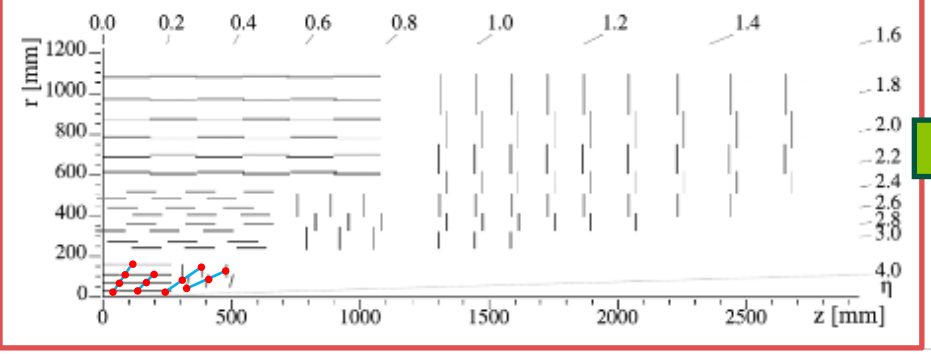
- **Learn from Run-3 and try to include redundancies early on**

- **Extra redundancies already present**
  → HLT baseline involves two seeding algorithms:
  - **Patatrack**: Extension to the first layers of the OT + allowance to skip layers
    → cleaner seeding from the start
    → **extension to more hits/tracker parts** upfront **reduces risk of** having to rely on **low-multiplicity recoveries** (e.g. doublet recovery in Run-3)
  - **LST**: **Full** trackers seeds covering both prompt and displaced efficiency
    → unaffected by broken IT
    → skipping layers: work in progress

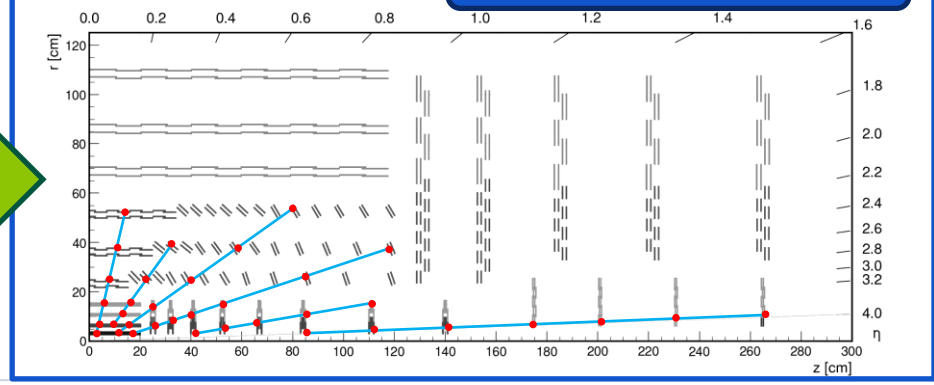


Example: Patatrack Tracks (Seeding)

Start of Run-3



Current Phase-2 baseline



Ready for failure scenarios?



Ready for failure scenarios?



→ Need tests to answer those questions!

- Samples for **realistic tracker failure scenarios**
- If simple to provide, even **stress-tests (sector holes, especially in first IT/OT barrel layers)** samples are OK for preliminary algorithm robustness tests
- See Suchandra's talk and Stef's overview

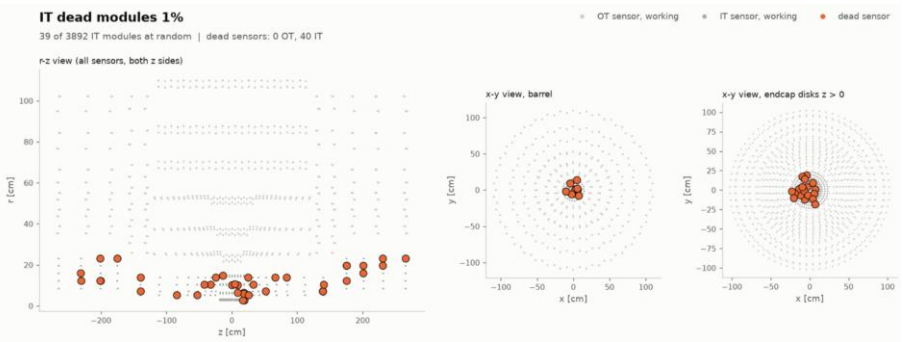
Ready for failure scenarios?

- How well does this already work?
- How difficult would it be to mitigate a realistic failure scenario?
- Could we handle it on the configuration-level or are algorithmic developments required?

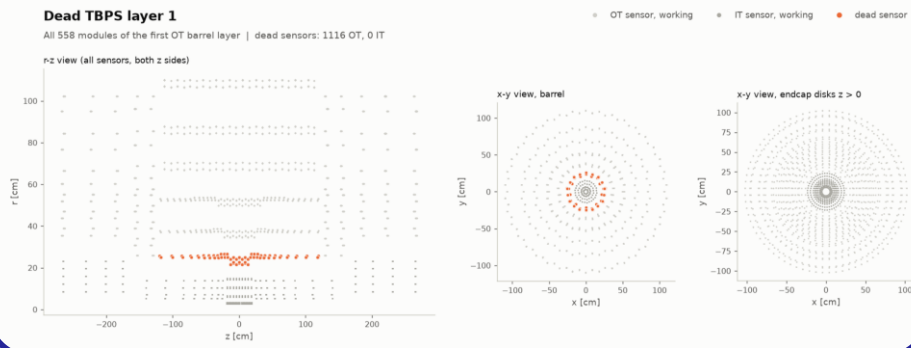
→ Need tests to answer those questions!

- Samples for **realistic tracker failure scenarios**
- If simple to provide, even **stress-tests (sector holes, especially in first IT/OT barrel layers)** samples are OK for preliminary algorithm robustness tests
- See Suchandra's talk and Stef's overview

For example: **IT dead modules 1%**



Or: **Dead TBPS layer 1**



Conclusions

Goal: same or better physics at lower timing, using the full Phase-2 tracker

New baseline:

Patatrack → LST → mkFit → CKF → DNN

Higher efficiency (including displaced tracks!)**, fewer fakes+duplicates,** and **huge time reduction** (4.9 s → 1.1 s / evt)

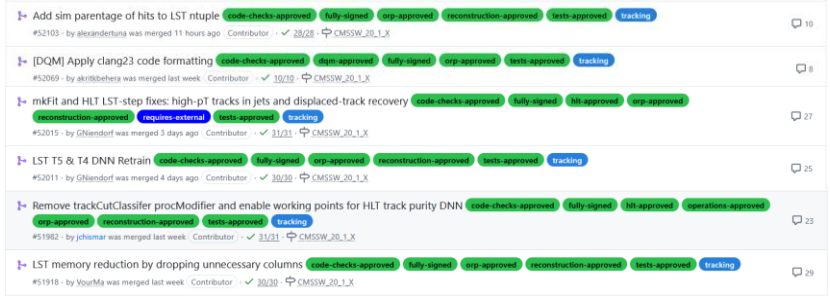
Moving fast: continues progress by many teams on LST, mkFit, ITACA, ACTS… Coming PRs on improving physics and timing/memory performance + heterogeneous ports

Run-3 lessons: redundancy is built in early (two seeders, OT extension). We **need failure-scenario samples** to test how robust the chain really is

CMSSW PRs opened last week



CMSSW PRs merged last week

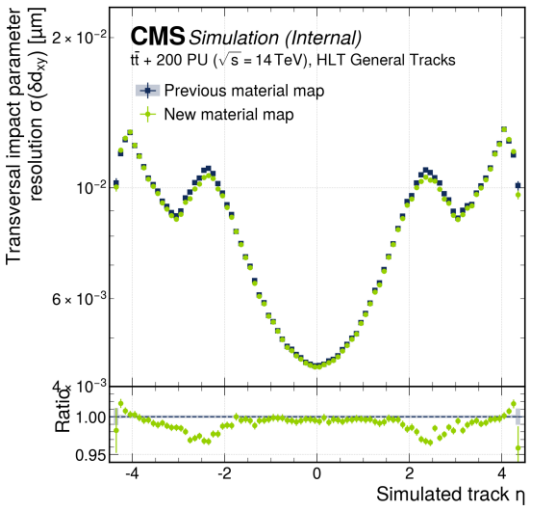


Backup

More details on everything

Backup

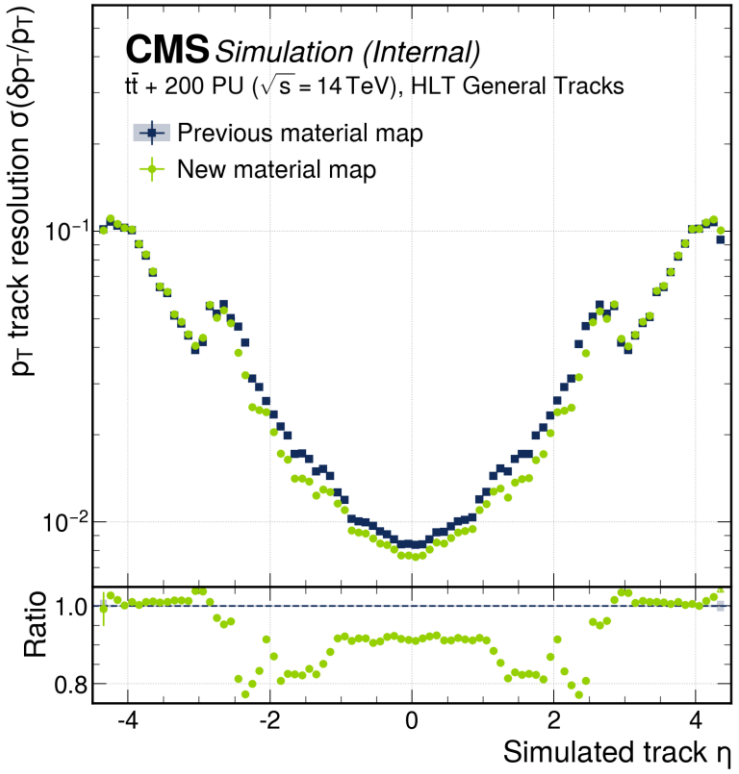
New material map



- **Why it matters?**
- The Combinatorial Kalman Filter (CKF) track fit models each module as a thin layer of material (x/X_0 for multiple scattering, ξ for energy loss). These values come from a material map, trackerRecoMaterial.xml.

- **Previous map:** From the tkLayout material model (IT711, May 2023)
- 27 values: one per layer or disk

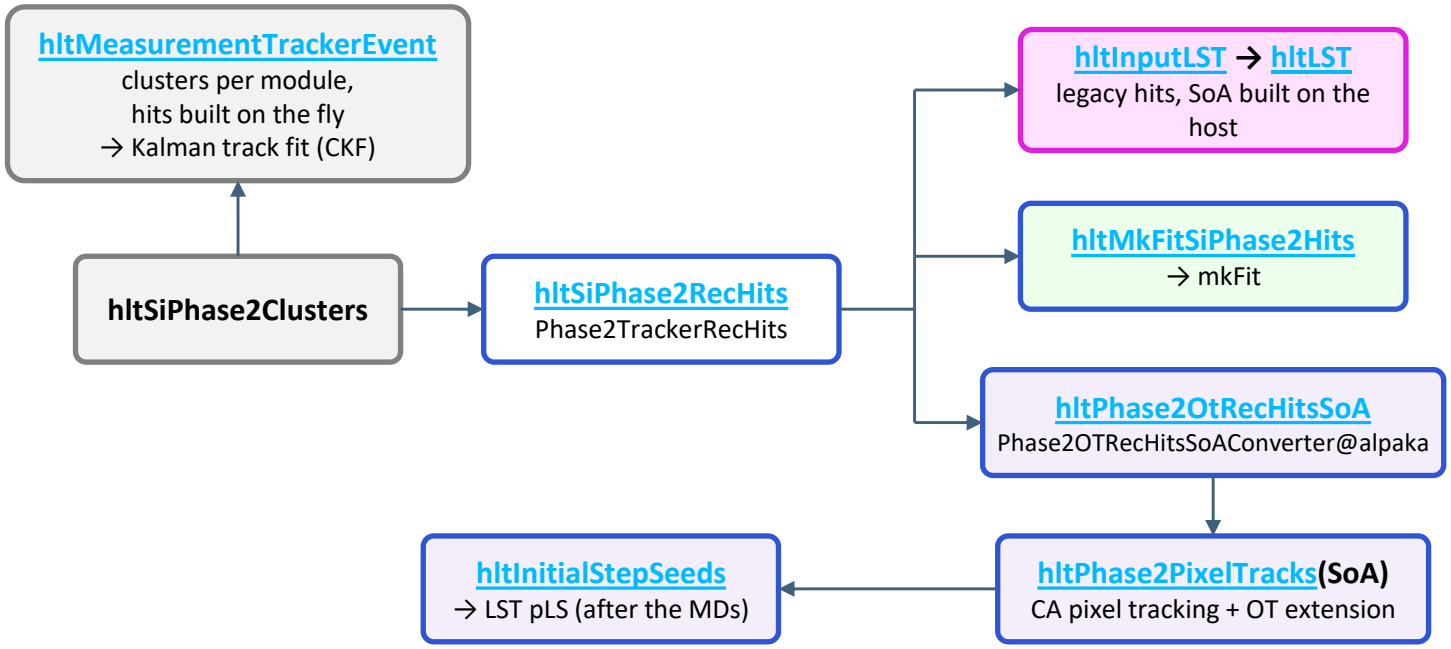
- **New map in CMSSW:**
- **Geant4 scan** (**trackingMaterialProducer10GeVNeutrino_ForPhaseII.py**): 200k ν_μ (p_T = 10 GeV, $|\eta| < 4$) used as non-interacting, straight-line probes: each step records x/X_0 and ξ of the material it crosses, plus the track angle at each sensor
- **Assignment and projection** (**TrackingMaterialAnalyser.cc**): material is assigned to the nearest sensor along the track
- It is scaled by cosθ, giving the value a perpendicular track would see
- **Grouping and averaging**: 404 groups (v3), one per sensor, plus Flipped and Unflipped Inner Tracker rods (Layers 1–2) kept separate in ϕ
  - Per group: mean x/X_0 and ξ over the tracks that cross it
  - Output: trackerRecoMaterial.xml
- **Use in reco:** each module gets its group's values (*MediumProperties*)
- The CKF applies the real crossing angle for multiple scattering and energy loss



Long term next step for Kalman Filter algorithms? ACTS (discussed in the Breno's contribution)

Who reads the OT RecHits (HLT)

Patatrack | LST | mkFit | CKF | DNN

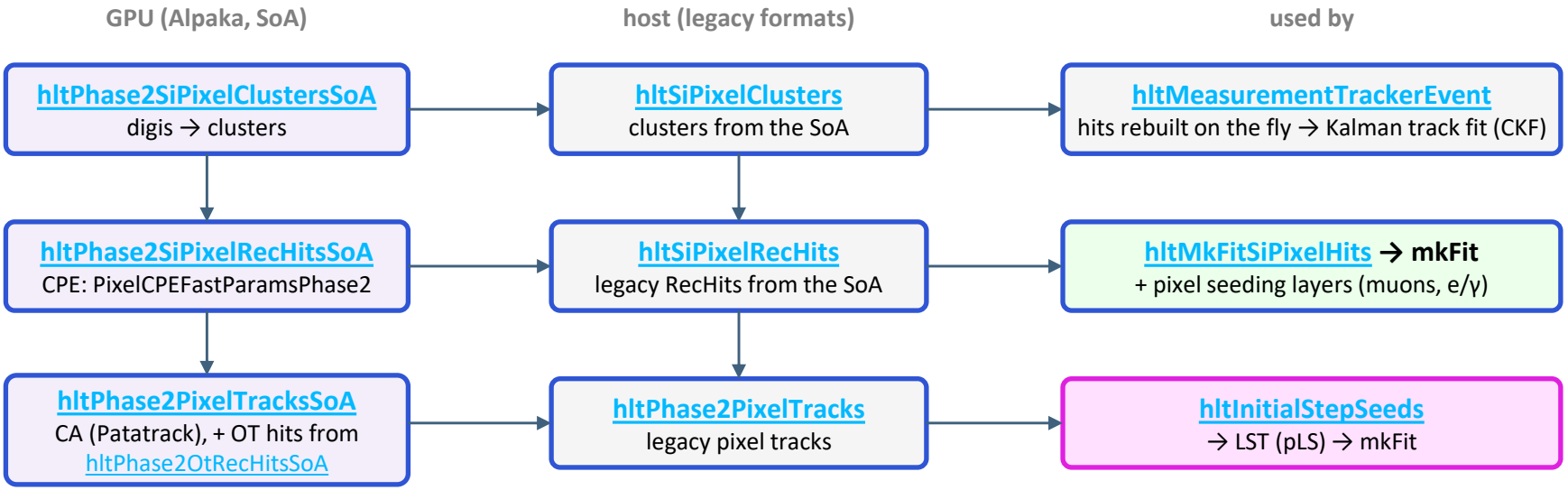


- **LST reads the legacy RecHits directly** and builds its own SoA; it does *not* use the OT RecHit SoA
- The OT SoA feeds the Alpaka CA pixel tracking with OT extension, which reaches LST only through the seeds
- The Kalman track fit (CKF, hltInitialStepTracks) rebuilds the hits from the clusters (MeasurementTrackerEvent)

HLTrigger/Configuration/python/HLT_75e33 at cms-sw/cmssw@f6063aa

Who reads the IT RecHits (HLT)

Patatrack → LST → mkFit → CKF → DNN



- Pixel clusters, RecHits and pixel tracks are built on the GPU (Alpaka SoA); legacy copies feed the CPU modules
- **LST never reads pixel hits directly**: they enter only through the seeds (pLS)
- mkFit reads the legacy pixel RecHits; the Kalman track fit (CKF) rebuilds them from the clusters

HLTrigger/Configuration/python/HLT_75e33 at cms-sw/cmssw@f6063aa

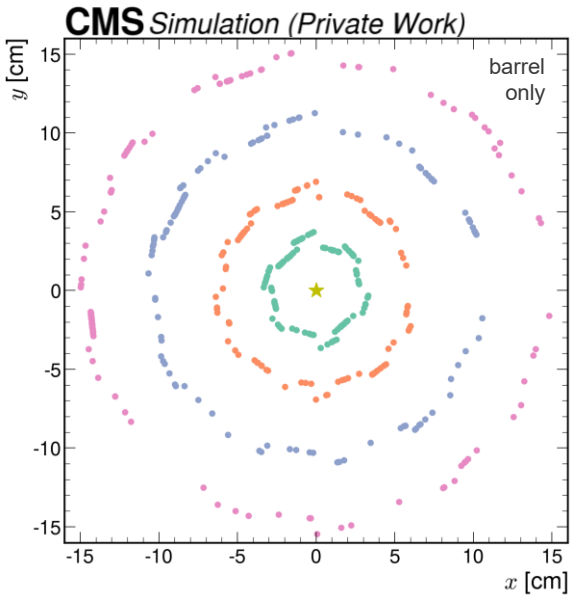
Backup - Patatrack

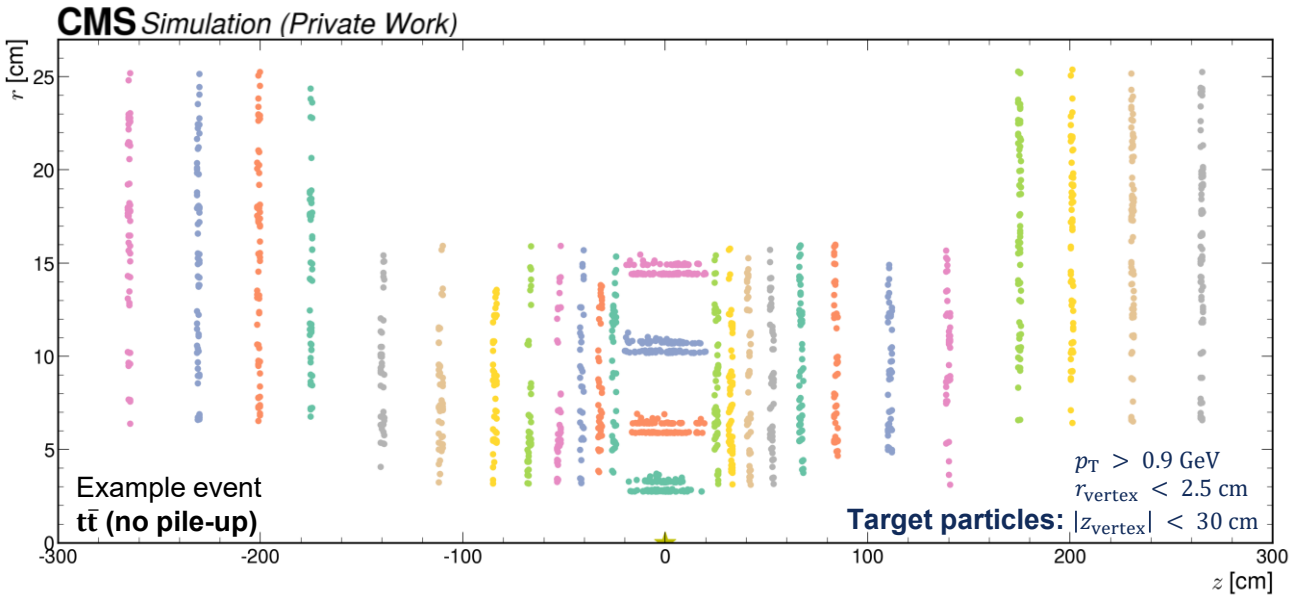
More details on the algorithm and configuration

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

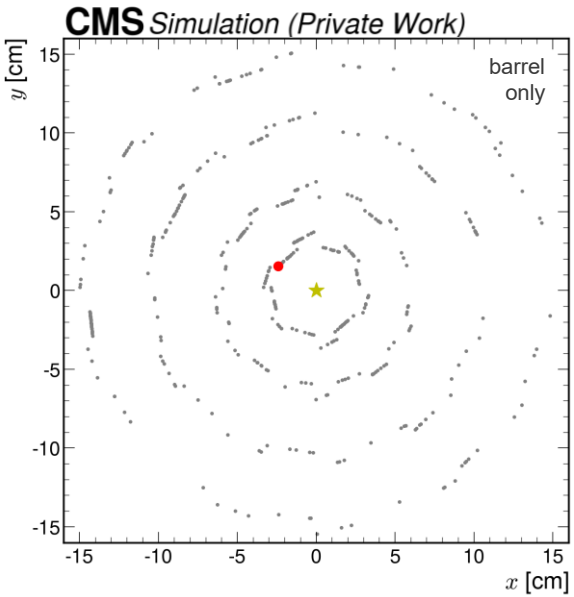
- IT RecHits + OT Pixel Hits

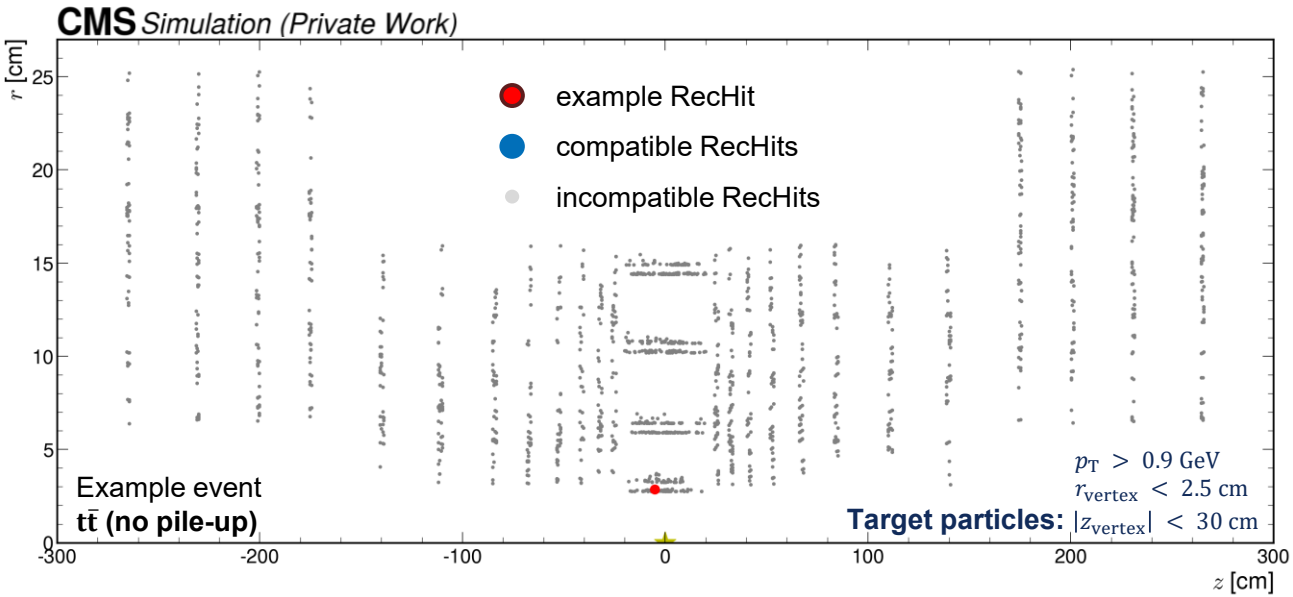
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

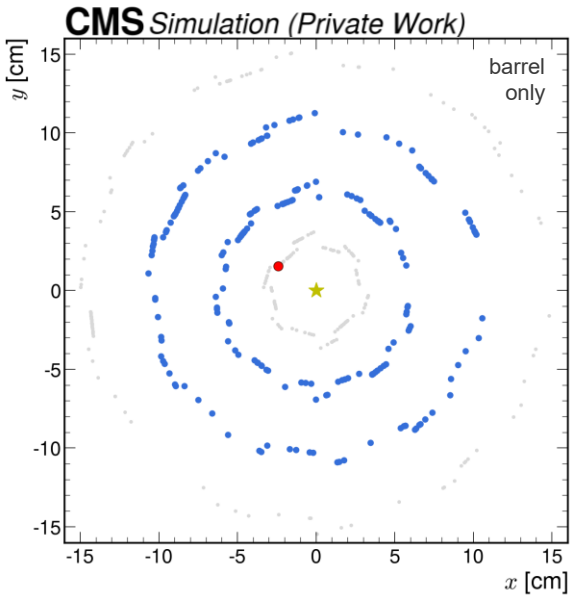
- Hit doublets

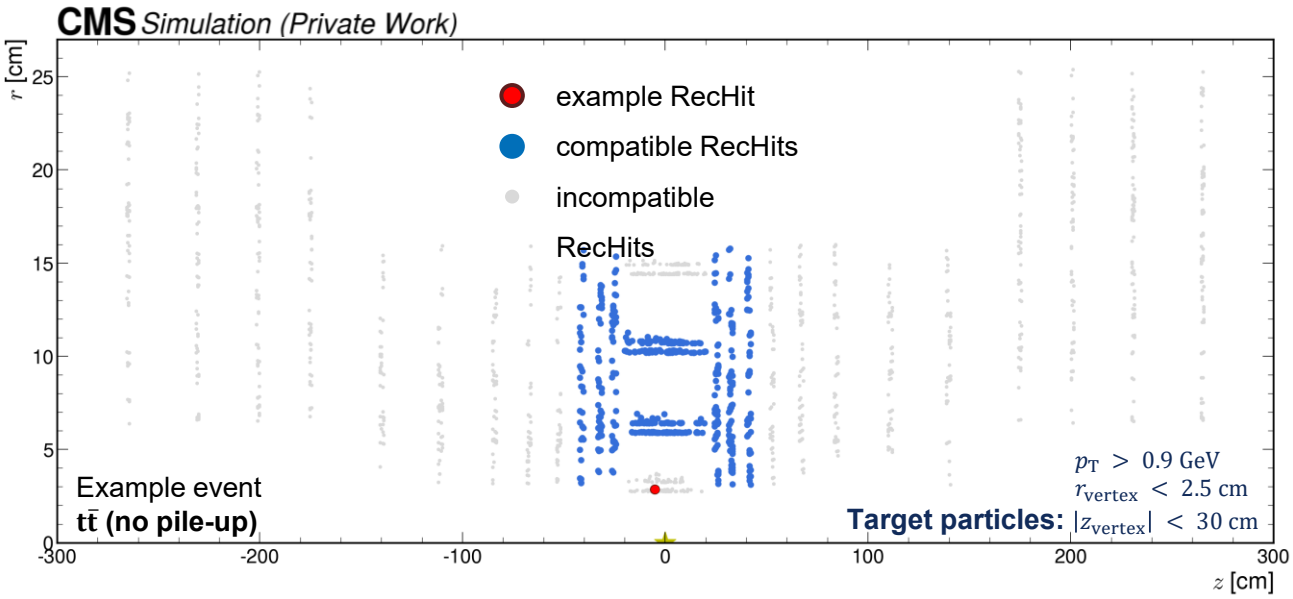
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

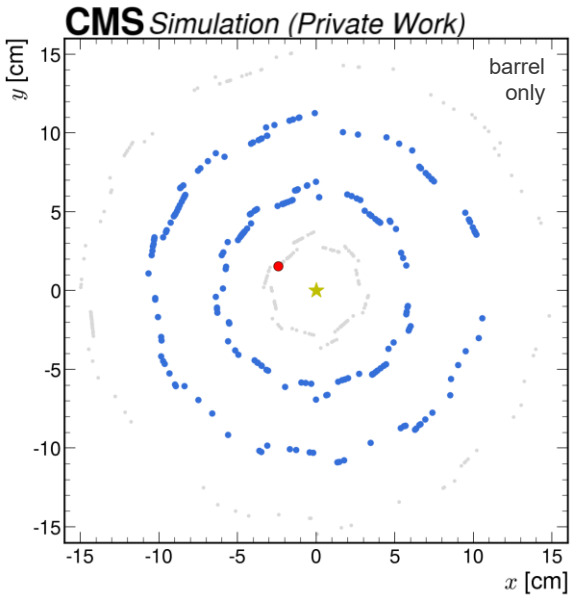
- Hit doublets

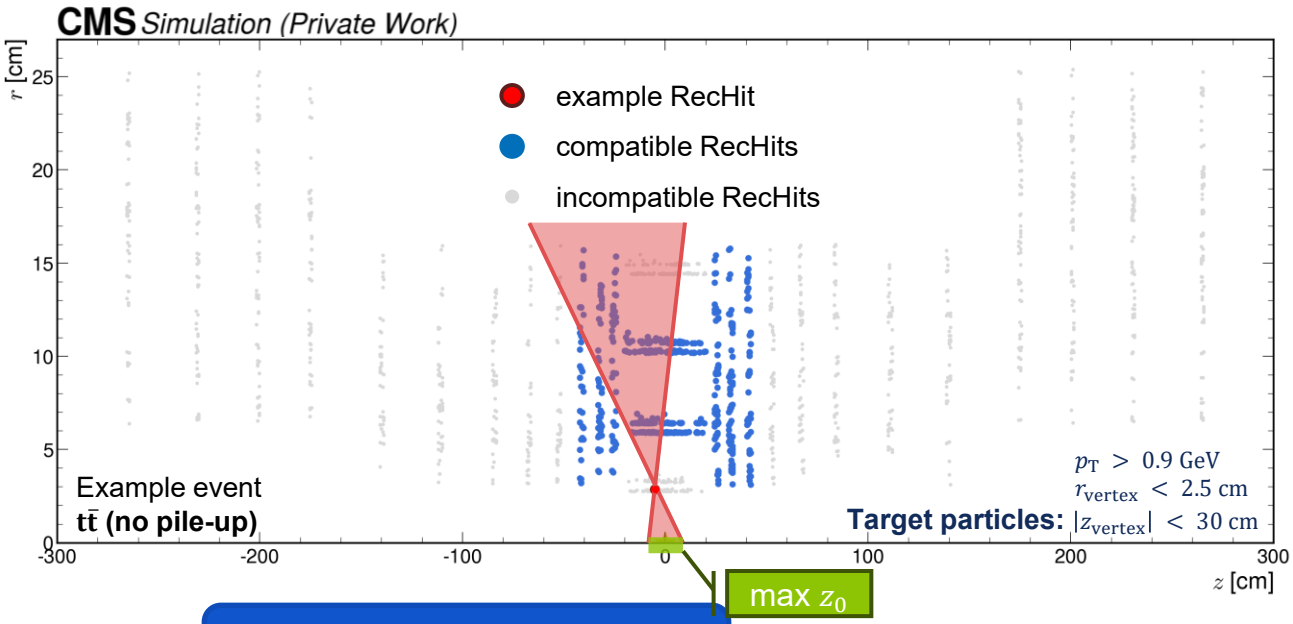
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

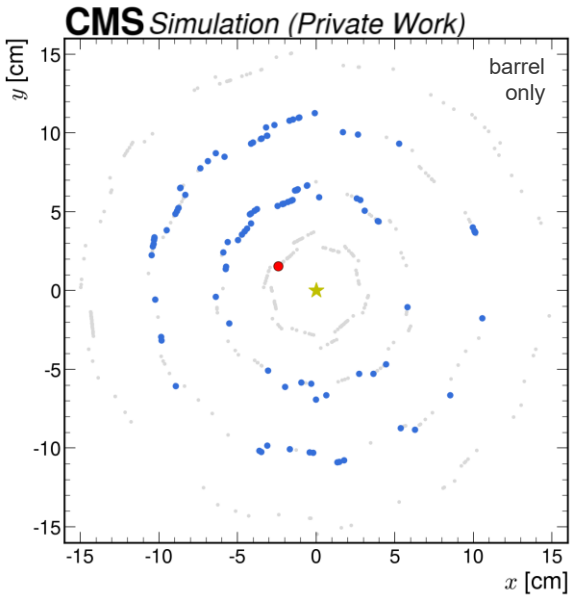
- Hit doublets

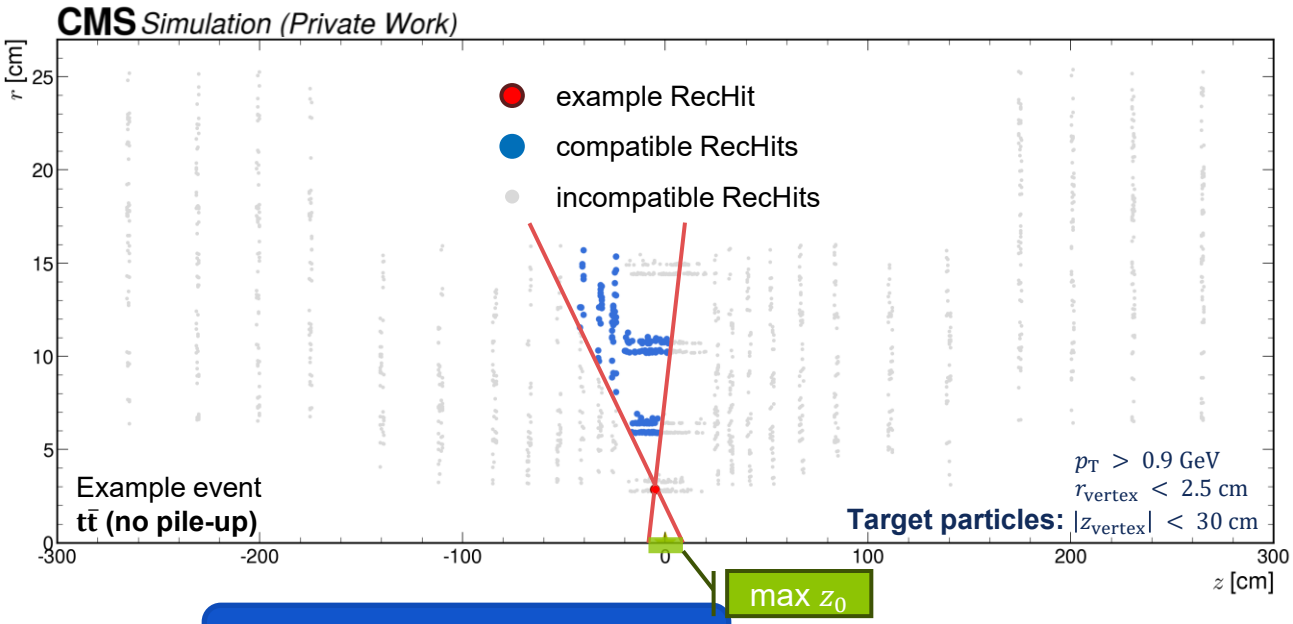
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

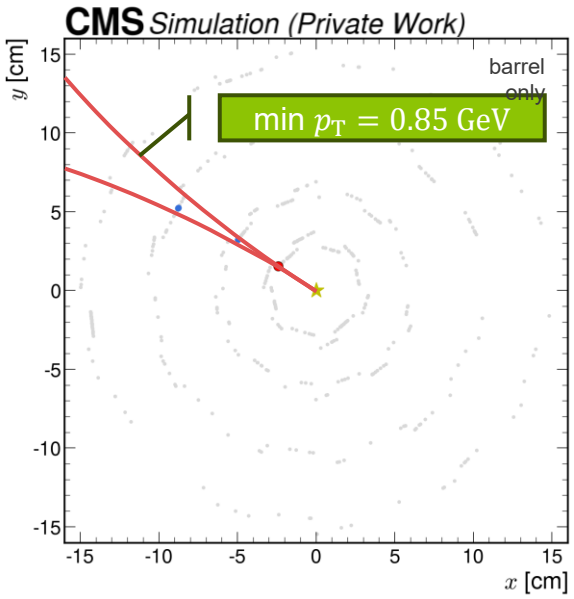
- Hit doublets

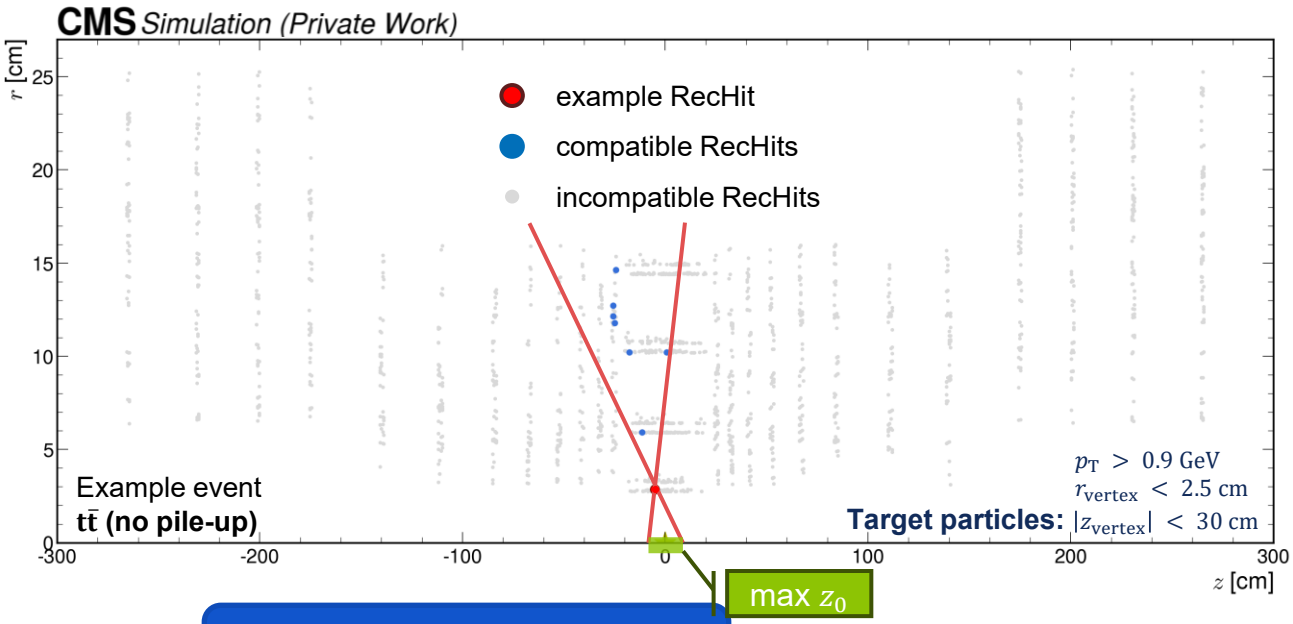
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

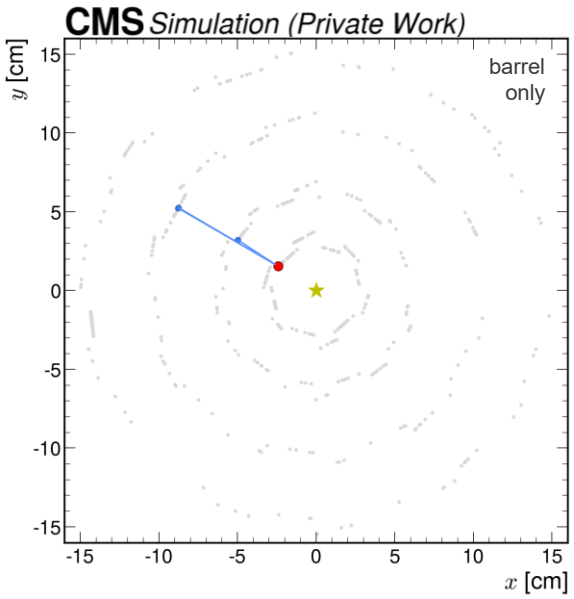
- Hit doublets

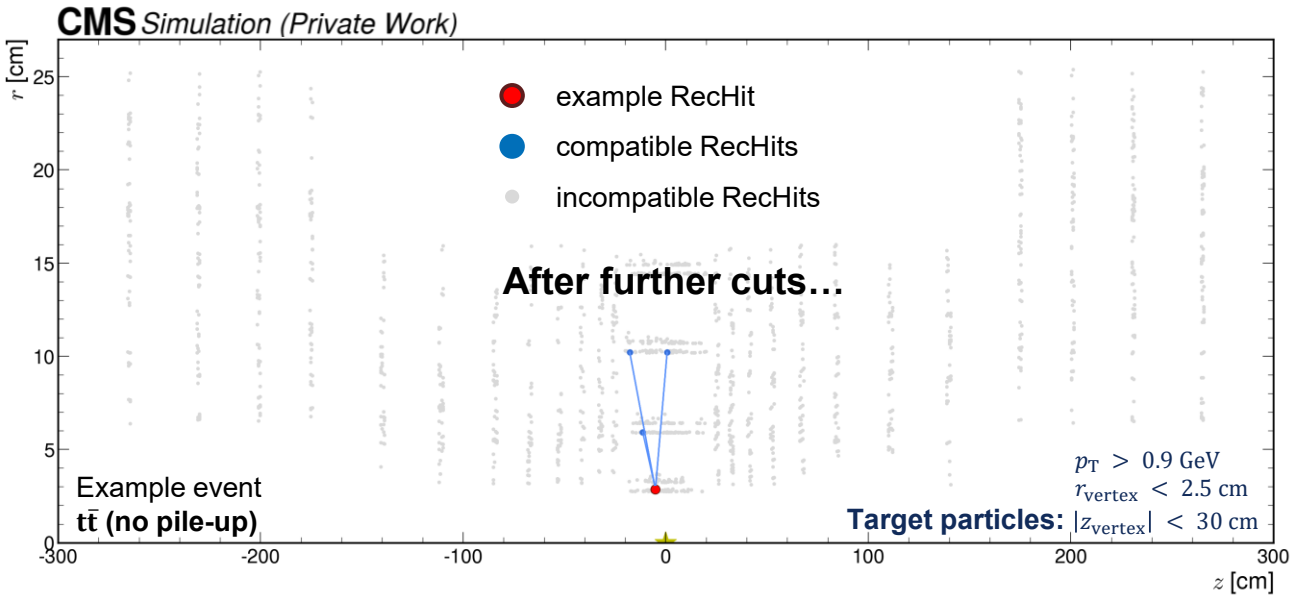
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- IT RecHits + OT Pixel Hits

Output

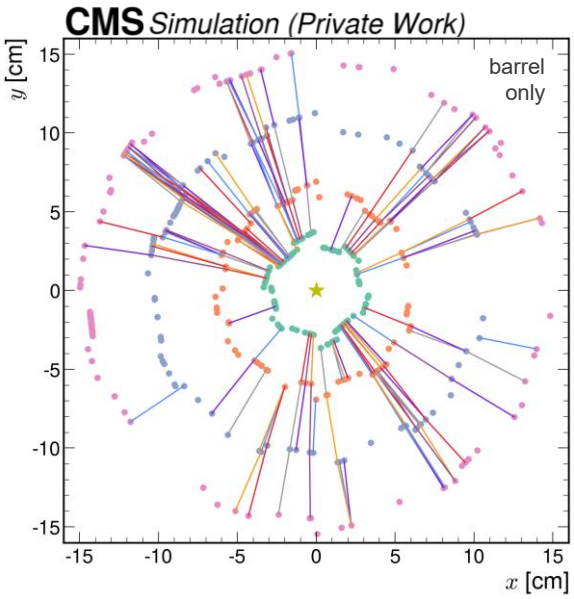
- Hit doublets

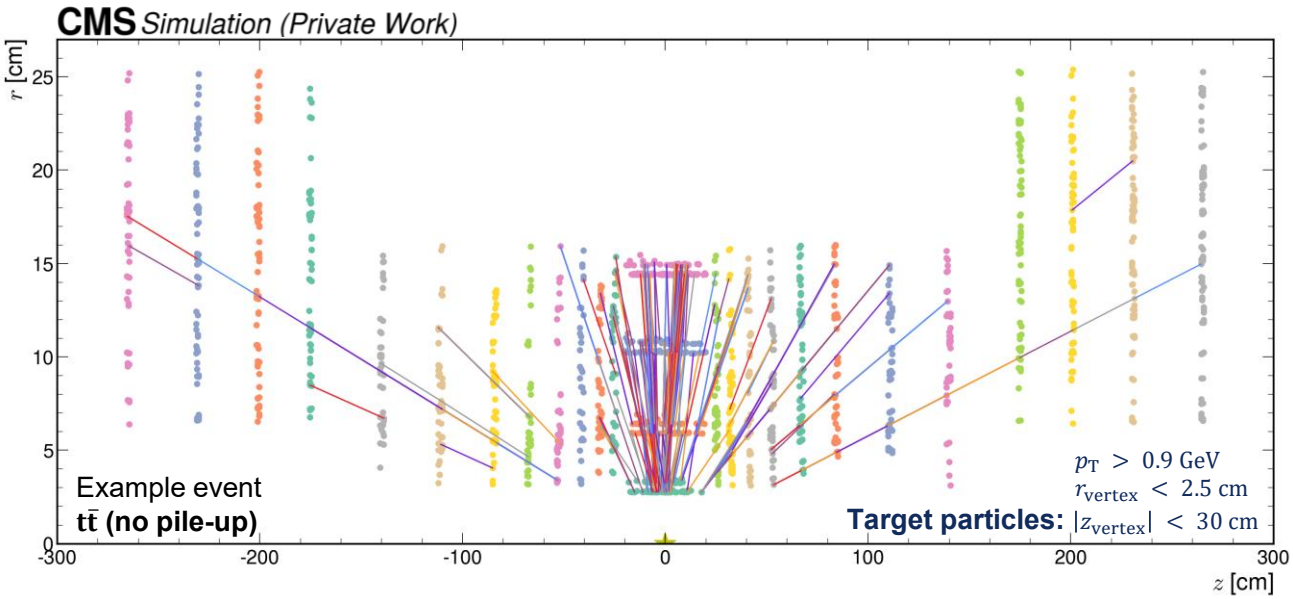
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- Doublets

Output

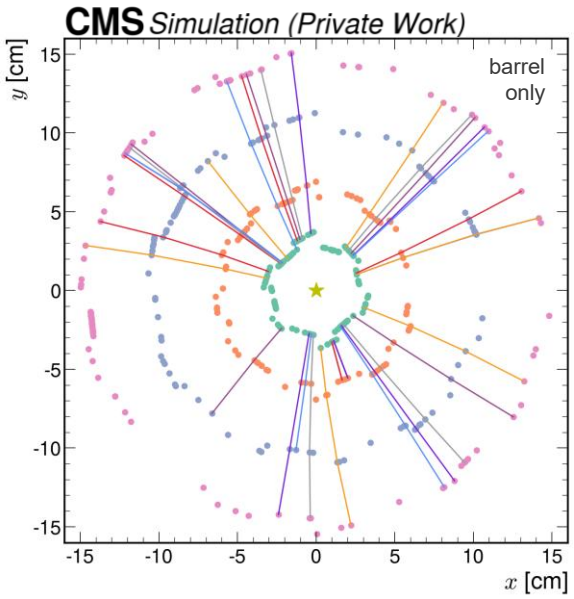
- Connected+compatible doublets

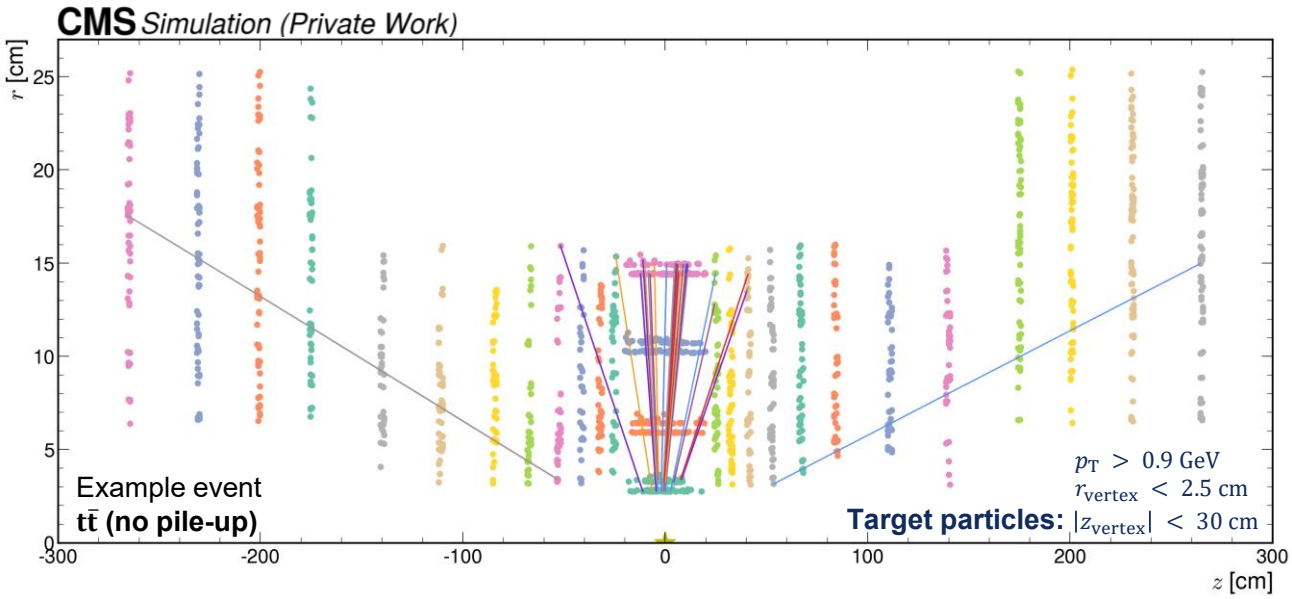
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- Doublet neighbors (triplets)

Output

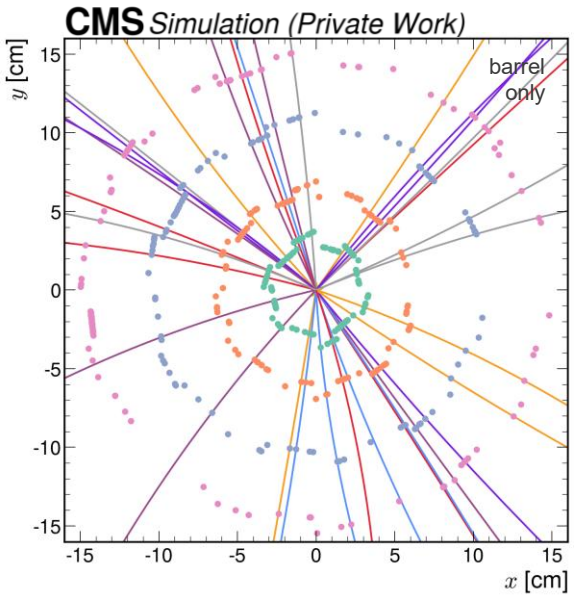
- N-tuplets

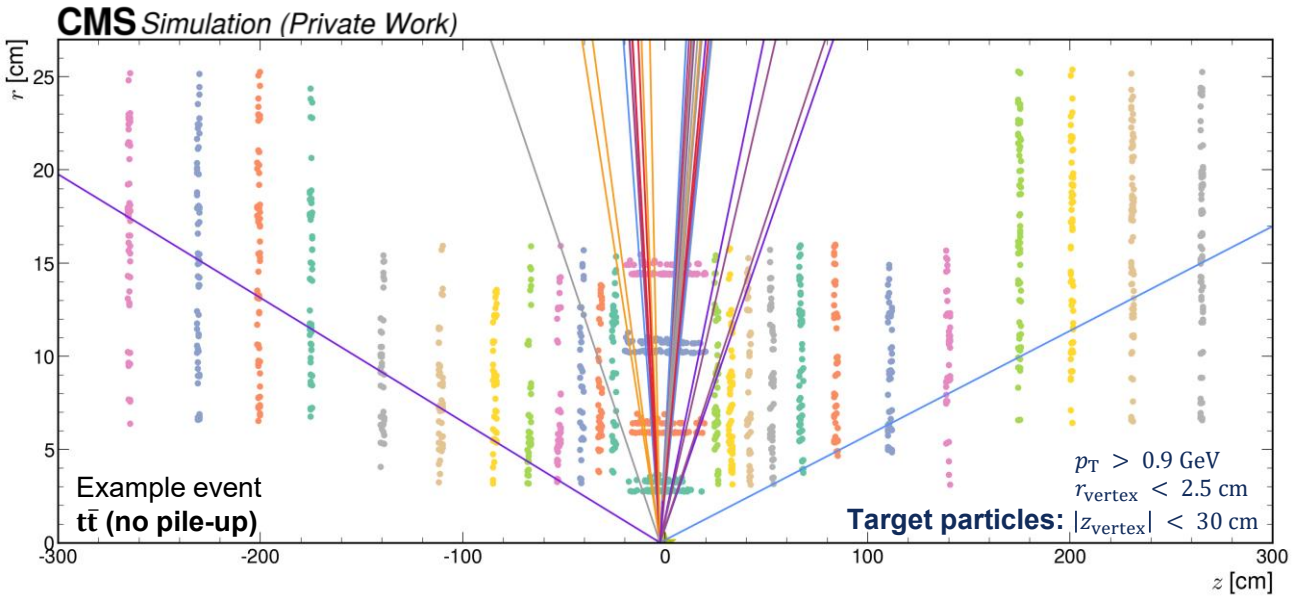
Code: cms-sw/cmssw (PixelSeeding)

Patatrack

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka | RecHits | Doublets | Triplets | N-tuplets | BL Fit | DNN





Input

- N-tuplets of IT+OT hits

Output

- Fitted tracks

Code: cms-sw/cmssw (PixelSeeding)

Backup - LST

More details on the algorithm and configuration

LST Mini-Doublets (MD)

Patatrack | LST | mkFit | CKF | DNN

GPU | Alpaka: MD | Line Segments | T3 | T4 / T5 | pT3 / pT5 | track cands.

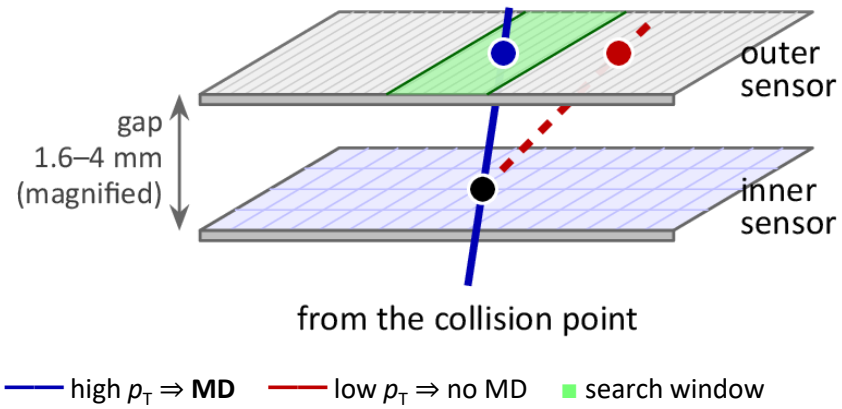
Input

- LST input hits (LSTInputSoA)
  → OT RecHits (Phase2TrackerRecHit1D)
  x, *y*, *z*, detId, cluster size
  HLT: hltSiPhase2RecHits
- Module conditions (LSTESData)
  PS/2S, layer, tilt, strip direction
- Pixel seeds: *not* used

Output

- MDs (MiniDoubletsSoA)
  anchor + outer hit, Δϕ, $\Delta\phi_{change}$, Δ*z*
  → Line Segments (LS)
- **45 600** MDs / event

- **Mini-Doublet (MD) ≔ one hit on the lower and one on the upper sensor of the same PS/2S module** (1.6–4 mm apart): the first LST object
  - anchor hit: macro-pixel hit (PS), lower-sensor hit (2S)
  - every (lower, upper) pair of a module tested in parallel: **1 GPU thread per pair**
  - **same principle as the L1 track-trigger stubs** (p_T modules), here in software from all read-out hits, p_T > 0.8 GeV (L1 stubs: ≈ 2 GeV, in the front-end chips)



CPU: OT RecHits → hltInputLST (host SoA)
copy H→D
GPU: hits per module, count MDs (all pairs)
n_{MD}, sync
CPU: allocate exactly n_{MD} rows
GPU: create MDs (same cuts) → LS

Code: cms-sw/cmssw@f6063aa (MiniDoublet.h, Hit.h, LSTInputProducer.cc)

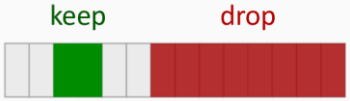
LST Mini-Doublets: selection

Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | T3 | T4 / T5 | pT3 / pT5 | track cands.

Applied in this order to every (lower, upper) hit pair; failing one step rejects the pair:

① Cluster size



cluster width ≤ **16** strips (both hits)

wider: delta rays, inclined tracks

② Relative position

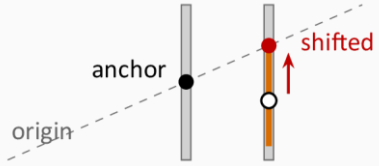


barrel: $|\Delta z| < \mathbf{2} / \mathbf{10}$ cm

endcap: $|\Delta z| < 1$ cm, $|\Delta r| < 2 / 10$ cm

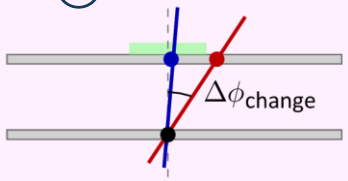
(PS / 2S)

③ Shift the strip hit



strip hit moved onto the line origin–anchor (tilted, endcap)

④ Small bend



$|\Delta\phi|, |\Delta\phi_{change}| < \theta_{cut}$

$\Leftrightarrow p_T > \mathbf{0.8}$ GeV

Bend threshold

$\theta_{cut} = \arcsin(\min(r\,k / p_T^{cut}, 0.95)) + \sqrt{(\theta_{MS}^2 + \theta_{PV}^2 + [\theta_{tilt}^2] + [\theta_{lum}^2])}$

first term: bend of a p_T^{cut} track (90–99% of θ_{cut}); k = 0.0057 GeV/cm

flat barrel	**L1**	**L2**	**L3**	**L4**	**L5**	**L6**
θ_{cut} [mrad]	199	283	395	524	671	893

Allowances & result

- θ_{MS} multiple scattering, θ_{PV} beam spot (0.1 cm/R, all)
- θ_{tilt} macro-pixel length (tilted modules)
- θ_{lum} vertex spread ±15 cm (endcap only)
- hits not beam-spot corrected; gap s from a table

45 600 MDs/event **64.7%** same particle

93.2% efficiency $p_T > 0.8$ GeV (PS 92.4%, 2S 94.5%)

PU200 $t\bar{t}$, D121, 332 events, standalone LST, HLT settings. Code: cms-sw/cmssw@f6063aa MiniDoublet.h

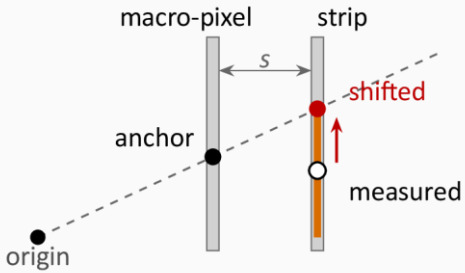
LST Mini-Doublets: parallax

Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | T3 | T4 / T5 | pT3 / pT5 | track cands.

① Position along the strip

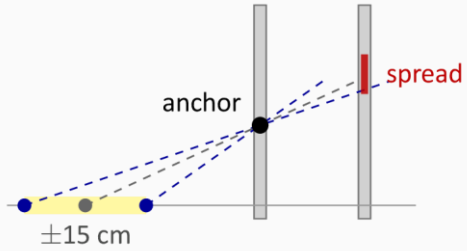
corrected: shifted hit (tilted, endcap)



$\Delta r = s \sin\theta / \sin(\theta+\beta)$
s from a table (1.6–4 mm), not the geometry

② Vertex spread along z

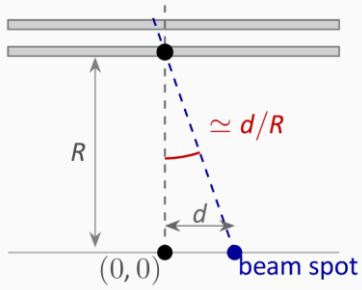
allowance: θ_{lum} (endcap only)



$\theta_{lum} = |\tan \Delta\phi| \cdot 15\ \text{cm} / |\Delta z|$ (~30 mrad)

③ Beam-spot offset in x–y

allowance: θ_{PV} (all modules)



$\theta_{PV} = 0.1\ \text{cm}/R$: covers $d \lesssim 1$ mm
hits are *not* beam-spot corrected

Where each term is used

	flat barrel	tilted barrel	endcap	
① shifted hit	– (not needed)	✓	✓	correction of the unmeasured coordinate
② θ_{lum} (vertex z)	– (not needed)	–	✓	tolerance in θ_{cut}
③ θ_{PV} (beam spot)	✓	✓	✓	tolerance in θ_{cut}
θ_{tilt} (macro-pixel length)	–	✓	–	tolerance in θ_{cut}

cms-sw/cmssw@f6063aa MiniDoublet.h

LST Mini-Doublets in the tracker

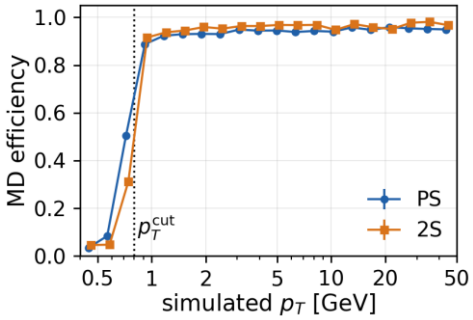
Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | T3 | T4 / T5 | pT3 / pT5 | track cands.

Per module type

Module type	MDs / event	same particle*	efficiency
PS barrel flat	5 000	65.7%	93.8%
PS barrel tilted	11 200	51.0%	93.4%
PS endcap	9 200	60.0%	89.2%
2S barrel	13 100	78.6%	95.8%
2S endcap	7 200	65.7%	92.6%
all	**45 600**	**64.7%**	**93.2%**

*both hits from the same simulated particle (signal or pileup); the rest are combinatorial pairs of hits from different particles



MD efficiency vs simulated p_T: turn-on at p_T^{cut} = 0.8 GeV, plateau ≈ 95%

What the MDs take from the tracker

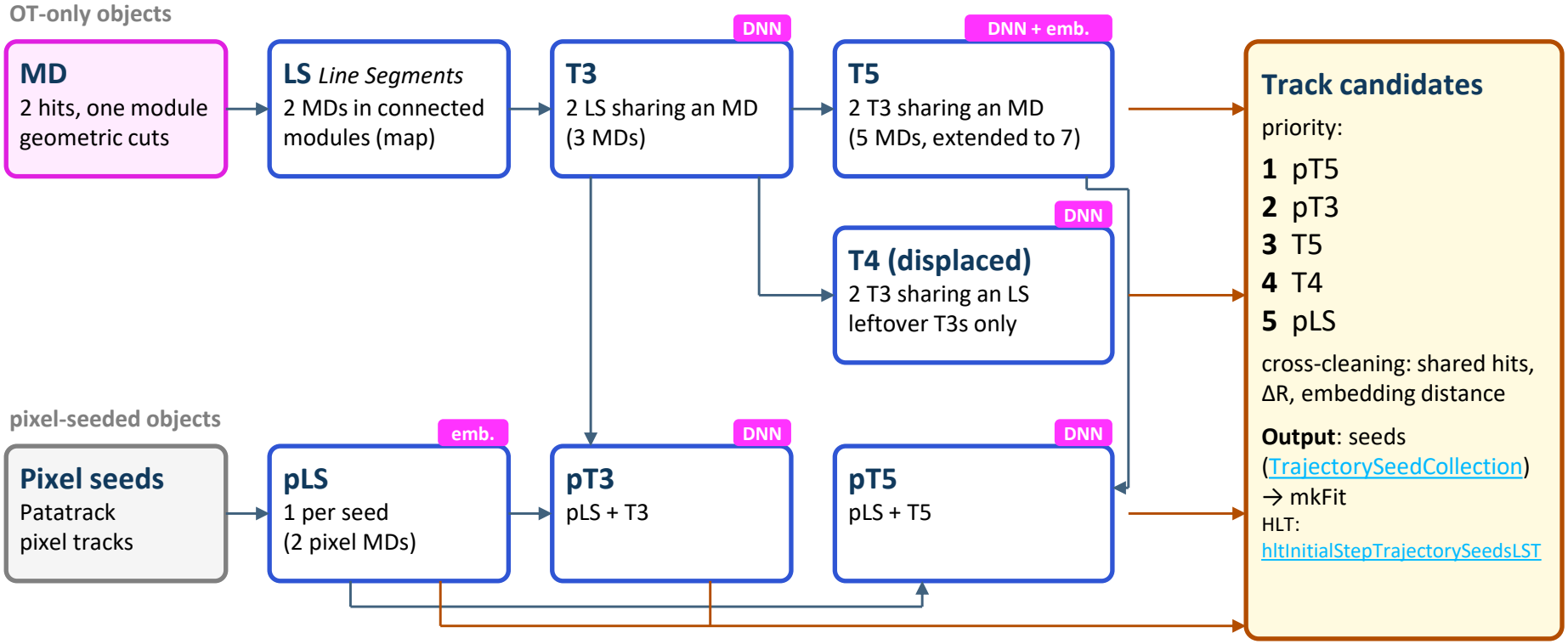
- **Hits**: OT RecHits from the CPE (Phase2StripCPEHLT); along the strip only the strip centre → shifted onto the line origin–anchor
- **Geometry**: module type, tilt *dr/dz*, strip direction from LSTGeometry; sensor gap *s* from a **hard-coded table**
- **No hit errors**: fixed windows, not χ^2
- **Beam spot**: hits not corrected; angles from (0, 0), tolerance θ_{PV} covers $\lesssim$ 1 mm
- **Tilted PS**: extra tolerance for the macro-pixel length along the tilted strip
- **Endcap**: luminous-region term θ_{lum} for the vertex spread along *z*

PU200 $t\bar{t}$, D121, 332 events, standalone LST, HLT settings (p_T^{cut} = 0.8 GeV); efficiency: signal particles, p_T > 0.8 GeV, $|\eta| < 2.4$

LST: from mini-doublets to tracks

Patatrack | **LST** | mkFit | CKF | DNN

MD → Line Segments → T3 → T4 / T5 → pT3 / pT5 → track cands.



Run order: MD → LS → T3 → T5 → pLS → pT5 → pT3 → T4 → track candidates
DNN: small MLPs in Alpaka; **emb.**: 6-dim embedding used for duplicate removal

cms-sw/cmssw@f6063aa RecoTracker/LSTCore/src/alpaka

LST Line Segments (LS)

Patatrack | **LST** | mkFit | CKF | DNN

MD | **Line Segments** | T3 | T4 / T5 | pT3 / pT5 | track cands.

Input

- MDs (MiniDoubletsSoA) of an inner module and of each connected outer module
- Module map (LSTGeometry) which modules a $p_T > p_T^{cut}$ track can connect

Output

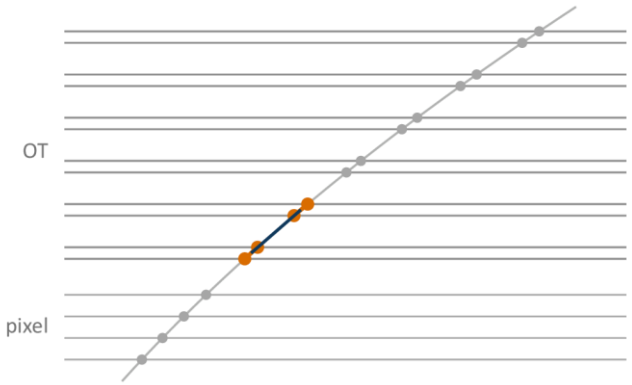
- LS (SegmentsSoA) 2 MDs + LS direction → T3
- pLS from pixel seeds are added to the same SoA

- **LS ≔ 2 MDs in modules connected by the module map**
  - every inner MD × every MD of each connected module: count (loose cuts) → allocate → create
  - cuts in order (barrel):
    - z window pointing back to the luminous region (±15 cm)
    - $\Delta\phi < \theta_{cut}$ ($p_T > p_T^{cut}$ + scattering + beam spot)
    - line residual: MD directions consistent with the LS (< 0.75 σ)
    - $\Delta\alpha$: each MD direction vs the LS direction
  - endcap: same z side, r window, $\Delta\phi$, line residual, $\Delta\alpha$

No DNN

No neural network and no duplicate removal for LS: geometric cuts only.

Per-module occupancy caps limit the number of LS.



cms-sw/cmssw@f6063aa Segment.h

LST Triplets (T3)

Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | **T3** | T4 / T5 | pT3 / pT5 | track cands.

Input

- LS pairs sharing the middle MD
- MD hit positions (for the circle and the DNN)

Output

- T3 (TripletsSoA) 3 MDs, circle radius + centre, charge, DNN scores, "loose pointing" flag
- → T5, T4, pT3

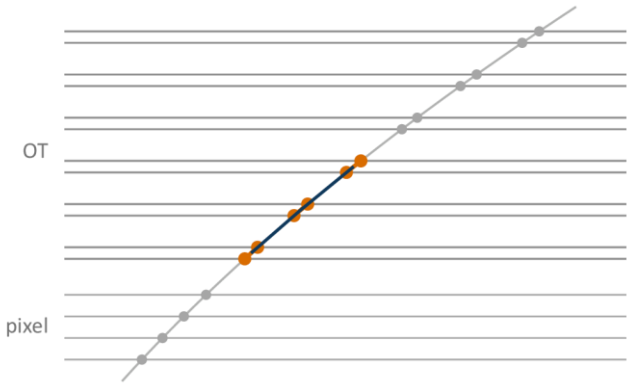
- **T3 ≔ 2 LS sharing an MD** (3 MDs, 6 hits)
  - cuts in order:
    - pointing: $\beta_{in} < \arcsin(\Delta r\, k/p_T^{cut}) + 0.02/\Delta r_{in}$; passing only at 1.7× ⇒ "loose pointing" (used only in T5)
    - *r*–*z* consistency: helix χ^2 per detector region
    - circle through the 3 anchors → radius, charge
    - **T3 DNN** (box below)
  - no duplicate removal

T3 DNN

14 inputs: MD positions, circle radius, β_{in}
→ softmax: fake / prompt / displaced

keep if P(prompt) *or* P(displaced) > working point in bins of p_T and η

MLP, 1 hidden layer of 32, in Alpaka



cms-sw/cmssw@f6063aa Triplet.h, NeuralNetwork.h

LST Quintuplets (T5) and Quadruplets (T4)

Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | T3 | **T4 / T5** | pT3 / pT5 | track cands.

Input

- T3 pairs
  T5: sharing an MD
  T4: sharing an LS
- T3 DNN scores, radii, MD directions

Output

- T5 (QuintupletsSoA), T4 (QuadrupletsSoA) hits, radius, DNN score, T5 embedding
- → pT5, track candidates

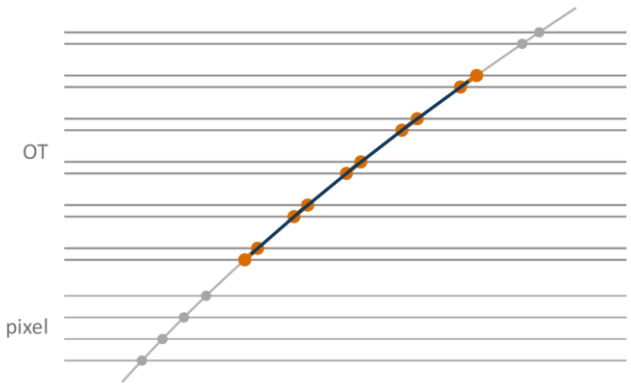
- **T5 ≔ 2 T3 sharing an MD** (5 MDs), from barrel L1–2 or endcap D1
  - dβ cuts, *r*–*z* χ^2, **T5 DNN**
  - extended up to 7 layers by merging T5s that share ≥ 8 hits
  - duplicates removed: shared hits (≥ 60% of the shorter) + embedding distance
- **T4 ≔ 2 T3 sharing an LS** (4 MDs), displaced tracks; barrel L ≥ 3 or endcap
  - built last: only T3s not used by T5 / pT5 / pT3
  - same charge, *r*–*z* χ^2, MD-direction pull < 6, **T4 DNN**

T5 / T4 DNN

T5: 35 inputs (incl. T3 scores, radii, MD-direction likelihood); keep if 1 − P(fake) > WP (95% of true T5s)

T5 embedding: 6-dim, for duplicates

T4: 33 inputs; keep if P(displaced) > WP (90% of displaced T4s)



cms-sw/cmssw@f6063aa Quintuplet.h, Quadruplet.h, Kernels.h

LST pixel-seeded objects (pLS, pT3, pT5)

Patatrack | **LST** | mkFit | CKF | DNN

MD | Line Segments | T3 | T4 / T5 | **pT3 / pT5** | track cands.

Input

- Pixel seeds (TrajectorySeedCollection) from Patatrack
  HLT: hltInitialStepSeeds
- T3 and T5 (OT objects)

Output

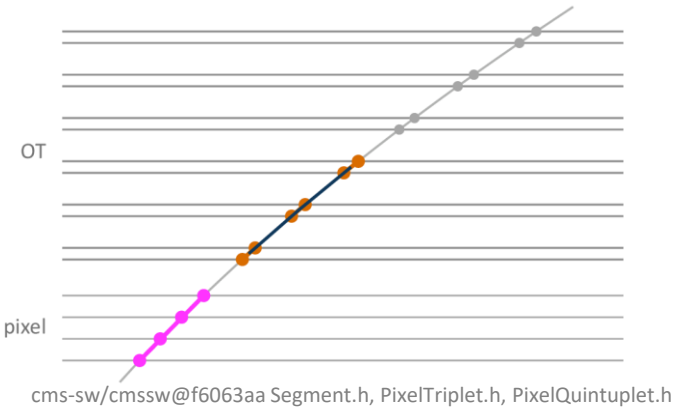
- pLS, pT3 (PixelTripletsSoA), pT5 (PixelQuintupletsSoA)
- → track candidates
- caps: 5000 pT3, 15000 pT5

- **pLS**: 1 per seed with $p_T > p_T^{cut} - 2\sigma$, stored as 2 pixel MDs
  - duplicates: $|\Delta\eta| < 0.1$ and ≥ 3 shared hits (skipped in the default HLT config, nopLSDupClean)
- **pT5 ≔ pLS + T5** (built first)
  - pT3-like cuts on the inner T3, *r*–*z* and *r*–ϕ χ^2 ($p_T < 5$ GeV), **DNN**
- **pT3 ≔ pLS + T3** (T3 from layer 1–2, PS)
  - 1/*R* overlap, pointing and β cuts, *r*–*z* χ^2 ($p_T < 5$ GeV), **DNN**

pT3 / pT5 DNN

pT3 DNN: 7 inputs → 1 score, two working points:
pT3 (95% signal efficiency)
pT5 (99–99.5%)

pLS embedding: 6-dim, to remove pLS duplicated by a T5



cms-sw/cmssw@f6063aa Segment.h, PixelTriplet.h, PixelQuintuplet.h

LST track candidates



Input

- pT5, pT3, T5, T4, pLS
  (after their own cleaning)

Output

- Track candidates
  → LSTOutputConverter
  → seeds
  (TrajectorySeedCollection)
  → mkFit
  HLT: hltInitialStepTrajectorySeedsLST
- cap: 10 000 OT-only + 300 000

- **Added in priority**, each only if not a duplicate of an earlier one
  - T5 vs pT5 / pT3: $|\Delta\eta|$, $|\Delta\phi| < 0.15$ and ≥ 4 shared OT hits
  - T4 vs T5: ≥ 3 shared hits; vs pT5 / pT3: ≥ 2
  - pLS vs T5: $\Delta R^2 < 0.02$ and small embedding distance; pLS vs pT3 / pT5: a shared pixel hit
  - tcpLSTriplets (HLT: on): triplet pLS also become candidates

DNN here

No new network: the **T5 DNN score** decides between duplicate T5s, the **pLS / T5 embeddings** remove pLS already covered by a T5.

The final HLT "DNN" step (chevron above) is a separate track classifier after the Kalman fit (CKF).

priority

1 pT5
2 pT3
3 T5
4 T4
5 pLS

cms-sw/cmssw@f6063aa TrackCandidate.h, LSTEvent.dev.cc

DNNs inside LST



Network	Object	Inputs → outputs	Used for
T3 DNN	T3	14 → 3 (fake / prompt / displaced)	keep T3 if prompt or displaced > WP
T5 DNN	T5	35 → 3	keep T5 if 1 − P(fake) > WP (95% of true T5)
T4 DNN	T4	33 → 3	keep T4 if P(displaced) > WP (90% of displaced T4)
pT3 DNN	pT3, pT5	7 → 1	two WPs: pT3 (95%), pT5 (99–99.5%)
T5 embedding	T5	30 → 6-dim vector	duplicate T5s (distance^2 ≤ 1), T5 extension
pLS embedding	pLS	10 → 6-dim vector	remove pLS duplicated by a T5

- All are small MLPs (1 hidden layer of 32) written directly in Alpaka: they run inside the LST kernels on GPU or CPU, no external inference library; weights in header files (NeuralNetwork.h).
- **MDs and LS use no DNN**: only geometric cuts.
- The HLT "DNN" step in the chain above is *not* part of LST: hltInitialStepTrackTorchClassifier (Torch track classifier on the final tracks; WP prompt 0.377, displaced 0.267).

cms-sw/cmssw@f6063aa RecoTracker/LSTCore/src/alpaka/NeuralNetwork.h, interface/alpaka/Common.h

Backup - mkFit

More details on the algorithm and configuration

mkFit: track building from LST seeds



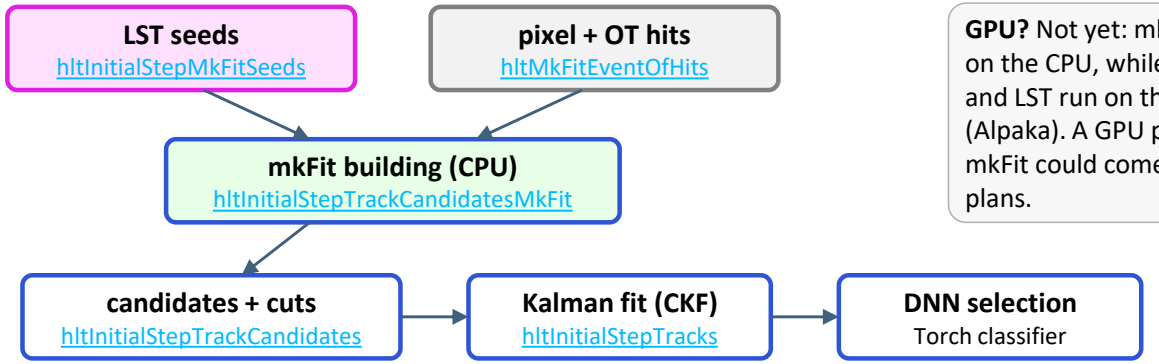
- **mkFit** ≔ **Kalman-filter combinatorial track finding** on the CPU
  - was already used in Run-3 HLT
  - **SIMD** (single instruction, multiple data): Matriplex processes 16 candidates at once (AVX-512)
  - **TBB** (Intel Threading Building Blocks, as in CMSSW): threads over η regions and seeds
  - each LST seed starts at the layer of its last hit: extended **outwards**, then **inwards**
  - **full tracker**: pixel + OT, each stacked-module sensor is its own layer

Input

- Seeds = LST track candidates (hltInitialStepTrajectorySeedsLST)
- Pixel + OT hits in mkFit format (hltMkFitEventOfHits)
- Geometry + iteration config (hltESPMkFit): 60 layers, mkfit-phase2-lstStep.json

Output

- Track candidates (hltInitialStepTrackCandidates)
  → Kalman track fit (CKF)
  → DNN selection



GPU? Not yet: mkFit runs on the CPU, while Patatrack and LST run on the GPU (Alpaka). A GPU port of mkFit could come in the plans.

cms-sw/cmssw@f6063aa HLTInitialStepSequence, RecoTracker/MkFit, MkFitCore

mkFit: building and cuts

Patatrack | LST | **mkFit** | CKF | DNN

CPU | TBB + SIMD — Hits | Seeds | **Building** | **Backward** | **Duplicates** | Output

For every seed, layer by layer (Kalman filter), then cleaning:

① Hit search

propagate the candidates to the next layer

window: ±3σ of the track error + margin, in ϕ and along the module

≤ 6 hits per candidate

② Hit acceptance

Kalman update: hit accepted if **$\chi^2 < 15$**

OT hits: cluster size < 8 and position along the strip compatible

③ Branching

keep the **5 best** candidates per seed (score: hits, holes, χ^2)

stop at > 4 holes or > 2 consecutive, or p_T < 0.9 GeV

④ Backward

Kalman fit inwards, inside mkFit (outliers χ^2 > 20 dropped, p_T > 5 GeV)

search again inwards: kept if it adds ≥ 2 pixel layers (1 if $|d_0|$ < 2 cm)

⑤ Duplicates

shared hits ≥ 24% of the shorter track

the track with pixel hits wins, otherwise the higher score

Output and final fit

- candidates with p_T > 0.85 GeV (0.8 for $|\eta|$ > 1.4) and ≥ 4 hits
- **Kalman track fit (CKF)**: **separate** CMSSW module (hltInitialStepTracks) that refits the candidates from scratch (mkFit errors ×100): fitter–smoother, χ^2 cut 20, ≤ 6 outliers
- **DNN selection**: score > 0.377 (prompt), > 0.267 if d_{xy} > 0.5 cm

Configuration

- mkfit-phase2-lstStep.json + overrides in hltESPMkFit: p_T^{min} 0.9 GeV, cluster size < 8, backward search ≥ 2 pixel layers
- seed cleaning off (LST seeds); clone engine; 5 η regions (barrel, transition ±, endcap ±)

MkFinder.cc, MkBuilder.cc, MkStdSeqs.cc; JSON values from cms-data RecoTracker-MkFit (main)

mkFit in the tracker

Patatrack → LST → **mkFit** → CKF → DNN

Hits → Seeds → **Building** → Backward → Duplicates → Output

Geometry: 60 layers

Sub-detector	layers	indices
Pixel barrel	4	0–3
OT barrel	6 × 2 sensors	4–15
Pixel disks (+*z*)	12	16–27
OT endcap (+*z*)	5 × 2 sensors	28–37
Pixel disks (−*z*)	12	38–49
OT endcap (−*z*)	5 × 2 sensors	50–59

built from TrackerGeometry (MkFitGeometryESProducer); every module stored with position, normal and strip direction for propagation to the module plane

Layer plan per η region

- **barrel**: pixel barrel → OT barrel
- **transition**: pixel barrel, pixel disks, OT barrel, OT endcap
- **endcap**: pixel barrel 1–2 → pixel disks

seeds are split into 5 regions (endcap −, transition −, barrel, transition +, endcap +) and processed in parallel

How the hits are used

- OT hits: 3D point + covariance; the strip / macro-pixel length enters only as the error along the strip
- **no stubs, no VectorHits**: the two sensors of a module are two consecutive layers
- pixel hits with cluster size; dead pixel modules from the quality DB
- **vs LST**: LST pairs the two sensors into MDs; mkFit searches them one at a time

MkFitGeometryESProducer, MkFitPhase2HitConverter, MkFitSiPixelHitConverter

mkFit: HLT modules



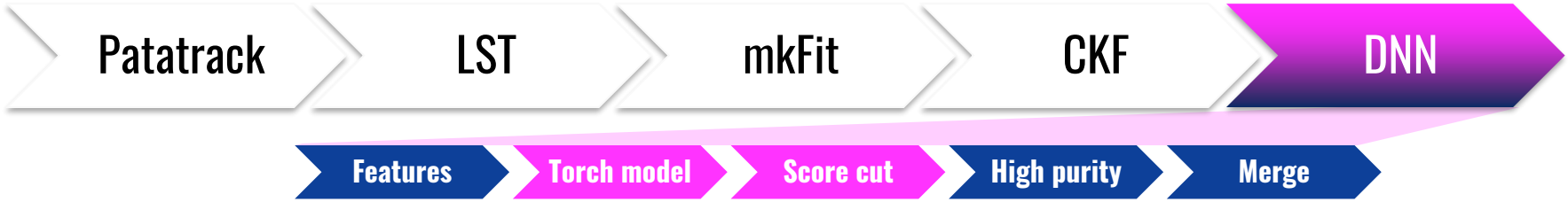
Module	Plugin	Inputs / key settings
hltMkFitSiPixelHits	MkFitSiPixelHitConverter	hltSiPixelRecHits, hltSiPixelClusters
hltMkFitSiPhase2Hits	MkFitPhase2HitConverter	hltSiPhase2RecHits, hltSiPhase2Clusters
hltMkFitEventOfHits	MkFitEventOfHitsProducer	hits + beam spot; pixel quality DB
hltInitialStepMkFitSeeds	MkFitSeedConverter	LST seeds (hltInitialStepTrajectorySeedsLST)
hltInitialStepTrackCandidatesMkFit	MkFitProducer	clone engine, duplicate removal, config hltESPMkFit
hltInitialStepTrackCandidates	MkFitOutputConverter	$p_T > 0.85$ GeV (0.8 for $\|\eta\| > 1.4$), ≥ 4 hits
hltInitialStepTracks	TrackProducer	Kalman fitter–smoother (CKF)
hltInitialStepTrackTorchClassifier	TrackTorchClassifier	DNN; WP 0.377 prompt, 0.267 displaced

HLTrigger/Configuration/python/HLT_75e33 at cms-sw/cmssw@f6063aa

Backup - DNN

More details on the track selection

Final DNN: track classifier



Input

- Fitted tracks (hltInitialStepTracks) from LST → mkFit → Kalman fit (CKF)
- Online beam spot (for d_{xy}, d_z)

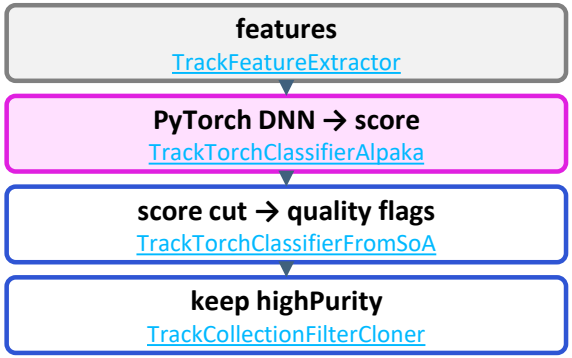
Output

- High-purity tracks (hltInitialStepTrackSelectionHighPurity)
- → hltGeneralTracks (merged, p_T > 0.8 GeV)

- **Track classifier ≔ PyTorch DNN** that scores every fitted track (genuine vs fake)
  - 15 features per track (box below); one score per track
  - inference through Alpaka + PyTorch (model.pt); CPU backend in the menu (alpaka_serial_sync)
- **Cut**: keep if score ≥ **0.377**, or if $|d_{xy}| > 0.5$ cm and score ≥ **0.267**
  - displaced tracks get a looser cut; tuned for 99.5% (config comment)
  - same cuts for loose / tight / highPurity; only highPurity tracks are kept

15 input features

- d_{xy}, d_z to the beam spot + errors
- η, ϕ + errors
- normalised χ^2, ndof
- lost inner / outer hits
- layers without measurement
- valid pixel hits, valid OT hits



a similar Torch selector also cleans the Patatrack pixel tracks (hltPhase2PixelTrackTorchHighPuritySelector)

cms-sw/cmssw@f6063aa HLTInitialStepHPSelectionSequence, RecoTracker/FinalTrackSelectors